xv6-k210

02_boot_trap — 启动/架构与 Trap/系统调用

`Q02_001` · `short_answer`

> 启动入口在哪里？（例如 linker.ld 的 ENTRY、 `_start` / `start` / `head` / `entry` 标签；必须给文件路径+符号证据）

项目	方法A	方法B
答案	链接脚本 `linker/linker64.ld` 定义 `ENTRY(_entry)` (第 2 行)。实际汇编入口有两个变体： `kernel/entry_k210.S` 定义 `_start` 符号 (第 2 行)， `kernel/entry_qemu.S` 定义 `_entry` 符号 (第 2 行)。两者都调用 `main` 函数进入 C 入口。	linker/linker64.ld 中 ENTRY(_entry)， kernel/entry.S 中 _entry 标签；k210 平台使用 kernel/entry_k210.S 中 _start 标签
证据	[1] `linker/linker64.ld` · ENTRY(_entry) `OUTPUT_ARCH(riscv) ENTRY(_entry)` [2] `kernel/entry_k210.S` · _start `.section .text.entry .globl _start _start:` [3] `kernel/entry_qemu.S` · _entry `.section .text .globl _entry _entry:`	[1] `linker/linker64.ld` · ENTRY `ENTRY(_entry)` [2] `kernel/entry.S` · _entry `.globl _entry _entry:` [3] `kernel/entry_k210.S` · _start `.globl _start _start:`

`Q02_004` · `short_answer`

> 模式切换涉及的关键寄存器/位是什么？（例如 RISC-V mstatus/sstatus、x86 cr0/cr4/eflags；必须给证据摘录）

项目	方法A	方法B
答案	RISC-V S 态关键寄存器： `sstatus` (SPP 位 bit 8 保存先前模式，SPIE bit 5 保存中断使能)， `satp` (SV39 页表基址)， `stvec` (陷阱向量基址)， `sepc` (异常返回地址)， `scause` (异常原因)。证据： `include/hal/riscv.h` 定义 `SSTATUS_SPP (1L << 8)` 、 `SSTATUS_SPIE (1L << 5)` 、 `SATP_SV39 (8L << 60)` 。 `kernel/trap/trap.c:usertrapret()` 清除 SPP 位返回用户态。	mepc (machine exception program counter), mstatus.MPP (machine previous privilege mode)
证据	[1] `include/hal/riscv.h` · SSTATUS_SPP `#define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User` [2] `include/hal/riscv.h` · SATP_SV39 `#define SATP_SV39 (8L << 60)` [3] `kernel/trap/trap.c` · usertrapret `x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode`	[1] `bootloader/SBI/rustsbi-k210/src/main.rs` · main `mepc::write(_s_mode_start as usize);` `mstatus::set_mpp(MPP::Supervisor);`

`Q02_006` · `short_answer`

> 从入口汇编/固件交接到 C/Rust 主入口函数的跳转链是什么？（列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	启动链：1) RustSBI 固件 (M 态) → 2) `kernel/entry_k210.S:_start` 或 `kernel/entry_qemu.S:_entry` (汇编入口，设置栈) → 3) `call main` 跳转到 `kernel/main.c:main()` (C 入口) → 4) `main()` 中初始化顺序： `cpuinit()` → `floatinithart()` → `consoleinit()` → `kvminit()` → `kvminithart()` → `trapinithart()` → `procinit()` → `scheduler()` 。证据： `kernel/entry_k210.S:10` 调用 `main` ， `kernel/main.c:35-97` 完整初始化序列。	RustSBI _start → main() (M-mode) → enter_privileged() → kernel _entry/_start → main() (S-mode)

项目	方法A	方法B
证据	[1] `kernel/entry_k210.S` · call main `# jump into main call main` [2] `kernel/main.c` · main `void main(unsigned long hartid, unsigned long dtb_pa) { ... cpuinit(); floatinithart(); consoleinit(); kvminit(); kvminithart(); trapinithart(); procinit(); scheduler(); }`	[1] `bootloader/SBI/rustsbi-k210/src/main.rs` · main `#[export_name = "_start"] fn main() -> ! { ... enter_privileged(...) }` [2] `kernel/entry_k210.S` · _start `call main` [3] `kernel/main.c` · main `void main(unsigned long hartid, unsigned long dtb_pa)`

`Q02_007` · `fill_in`

> 早期初始化 (Early Initialization) 各项状态（每项必须 implemented / stub / not_found + 证据路径，格式： 项目：状态 [路径]）： - BSS 清零 (BSS Clearing): ___ - 早期串口输出 (Early Serial/UART Output): ___ - 设备树解析 (Device Tree Blob parsing, DTB): ___ - 页表初始化时机 (Page Table Init): ___（在 MMU 启用前/后？）

项目	方法A	方法B
答案	BSS 清零 (BSS Clearing): implemented [linker/linker64.ld:53-56 .bss 段定义，链接器自动处理] 早期串口输出 (Early Serial/UART Output): implemented [kernel/console.c:consoleinit() + sbi_console_putchar 通过 SBI 调用] 设备树解析 (Device Tree Blob parsing, DTB): not_found [main.c 接收 dtb_pa 参数但未显式解析 DTB] 页表初始化时机 (Page Table Init): implemented [kernel/mm/vm.c:kvminit() 在 kvminithart() 之前，MMU 启用前建立映射]	BSS 清零 (BSS Clearing): implemented [bootloader/SBI/rustsbi-k210/src/main.rs] 早期串口输出 (Early Serial/UART Output): implemented [bootloader/SBI/rustsbi-k210/src/main.rs] 设备树解析 (Device Tree Blob parsing, DTB): implemented [bootloader/SBI/rustsbi-qemu/src/main.rs] 页表初始化时机 (Page Table Init): 在 MMU 启用前 [kernel/mm/vm.c]
证据	[1] `linker/linker64.ld` · .bss `.bss : { sbss_clear = .; (.sbss .bss .bss.) ebss_clear = .; }` [2] `kernel/console.c` · consoleinit `void consoleinit() { ... sbi_console_putchar ... }` [3] `kernel/main.c` · main `void main(unsigned long hartid, unsigned long dtb_pa) { ... }` [4] `kernel/mm/vm.c` · kvminit `void kvminit() { ... kvmmap(...) ... } // 在 kvminithart 之前调用`	[1] `bootloader/SBI/rustsbi-k210/src/main.rs` · main `r0::zero_bss(&mut _sbss, &mut _ebss); r0::init_data(...)` [2] `bootloader/SBI/rustsbi-k210/src/main.rs` · main `let serial = p.UARTHS.configure(115_200.bps(), &clocks);` [3] `bootloader/SBI/rustsbi-qemu/src/main.rs` · main `let dtb_pa = ...; if let Ok(dt) = DeviceTree::load(data)` [4] `kernel/mm/vm.c` · kvminit `kvminit() creates page table before kvminithart() enables MMU via w_satp()`

`Q02_010` · `short_answer`

> 构建系统如何选择目标平台/架构与入口文件？（Cargo features/Kconfig/Makefile 条件；必须引用配置证据）

项目	方法A	方法B
答案	通过 Makefile 的 `platform` 变量控制： `platform := k210` (默认) 或 `platform := qemu`。使用 `#ifdef QEMU` 条件编译区分平台。入口文件固定为 `kernel/entry.S`，但实际根据平台使用 `entry_k210.S` 或 `entry_qemu.S`。证据： `Makefile:1-2` 设置 platform 变量， `Makefile:28-29` 添加 `-D QEMU` 标志。	Makefile 中 platform 变量选择 k210 或 qemu，通过条件编译和不同入口文件

项目	方法A	方法B
证据	[1] `Makefile` · platform `platform := k210 # platform := qemu` [2] `Makefile` · QEMU flag `ifeq ($(platform), qemu) CFLAGS += -D QEMU endif`	[1] `Makefile` · platform `platform := k210 # platform := qemu ... ifeq ($(platform), k210) SRC += $K/hal/spi.c ... else SRC += $K/hal/virtio_disk.c` [2] `kernel/entry.S` · _entry `QEMU version entry` [3] `kernel/entry_k210.S` · _start `K210 version entry`

`Q02_014` · `short_answer`

> 是否支持多平台启动（StarFive VisionFive2/LoongArch/多板型）？（搜索 visionfive|jh7110|loongarch；有则描述差异入口与互斥关系；无则写未发现）

项目	方法A	方法B
答案	未发现多平台支持。代码仅支持 K210 和 QEMU virt 两种平台，通过 Makefile 的 platform 变量切换。搜索 visionfive、jh7110、loongarch 均无匹配结果。	未发现 visionfive/jh7110/loongarch 支持，但支持 k210 和 qemu 双平台
证据	—	[1] `Makefile` · platform `platform := k210 # platform := qemu ... ifeq ($(platform), k210) ... else ...` [2] `kernel/entry.S` · _entry `QEMU entry point` [3] `kernel/entry_k210.S` · _start `K210 entry point`

`Q02_015` · `short_answer`

> trap/异常向量入口在哪里？（trap_handler/trap_vector/__alltraps 等；必须给证据）

项目	方法A	方法B
答案	陷阱向量入口：内核态通过 `kernel/trap/kernelvec.S:kernelvec` (第 8 行)，用户态通过 `kernel/trap/trampoline.S:uservec` (第 15 行)。 `kernel/trap/trap.c:trapinithart()` 设置 `w_stvec((uint64)kernelvec)` 。异常处理函数为 `kernel/trap/trap.c:usertrap()` 和 `kerneltrap()` 。	kernelvec (内核态), uservec (用户态)
证据	[1] `kernel/trap/kernelvec.S` · kernelvec `.globl kernelvec kernelvec:` [2] `kernel/trap/trampoline.S` · uservec `.globl uservec uservec:` [3] `kernel/trap/trap.c` · trapinithart `w_stvec((uint64)kernelvec);`	[1] `kernel/trap/kernelvec.S` · kernelvec `.globl kernelvec ... kernelvec: ... call kerneltrap` [2] `kernel/trap/trampoline.S` · uservec `.globl uservec ... uservec: ... save user registers in TRAPFRAME` [3] `kernel/trap/trap.c` · trapinithart `w_stvec((uint64)kernelvec)`

`Q02_017` · `short_answer`

> TrapFrame/寄存器保存结构体定义在哪里？寄存器数量与字节数是多少？（必须引用结构体定义证据）

项目	方法A	方法B
答案	定义在 `include/trap.h:17-93` 的 `struct trapframe`。包含：整数寄存器 32 个 (ra,sp,gp,tp,t0-t6,s0-s11,a0-a7) + 浮点寄存器 32 个 (ft0-ft11,fs0-fs11,fa0-fa7) + fcsr 控制寄存器 = 共 65 个字段。总字节数：548 字节 (0-544 为寄存器，544-548 为 fcsr)。	include/trap.h 中 struct trapframe 定义，包含 32 个整数寄存器 + 32 个浮点寄存器 + fcsr，共 552 字节
证据	[1] `include/trap.h` · trapframe `struct trapframe { / 0 / uint64 kernel_satp; ... / 544 / uint64 fcsr; };`	[1] `include/trap.h` · trapframe `struct trapframe { uint64 kernel_satp; ... uint64 ra; ... uint64 ft0; ... uint64 fcsr; } - 68 fields × 8 bytes = 544 bytes + fcsr`

`Q02_020` · `short_answer`

> 选择一个具体 syscall（优先 sys_write），追踪：用户指令 → trap → 分发 → 实现体。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	sys_write 路径：1) 用户态 `ecall` 指令 → 2) `kernel/trap/trampoline.S:uservec` 保存上下文 → 3) `kernel/trap/trap.c:usertrap()` 检测 `EXCP_ENV_CALL` → 4) `kernel/syscall/syscall.c:syscall()` 通过 `syscalls[SYS_write]` 分发 → 5) `kernel/syscall/sysfile.c:sys_write()` 实现文件写入。证据：`trap.c:97-107` 系统调用分支，`syscall.c:212` 分发表索引，`sysfile.c` 实现写入逻辑。	sys_write: xv6-user/usys.pl (user stub) → kernel/trap/trap.c (usertrap) → kernel/syscall/syscall.c (syscall dispatch) → kernel/syscall/sysfile.c (sys_write implementation)
证据	[1] `kernel/trap/trap.c` · usertrap `if (cause == EXCP_ENV_CALL) { ... syscall(); }` [2] `kernel/syscall/syscall.c` · syscall `p->trapframe->a0 = syscalls[num];` [3] `kernel/syscall/syscall.c` · syscalls[] `[SYS_write] sys_write,`	[1] `xv6-user/usys.pl` · entry `entry("write"): li a7, SYS_write; ecall; ret` [2] `kernel/trap/trap.c` · usertrap `if (r_scause() == 8) { ... syscall(); }` [3] `kernel/syscall/syscall.c` · syscall `p->trapframe->a0 = syscalls[num];` [4] `kernel/syscall/sysfile.c` · sys_write `uint64 sys_write(void) { ... return filewrite(f, p, n); }`

`Q02_021` · `short_answer`

> 列出 5-10 个“高价值 syscall”（fork/exec/mmap/open/write 等）的实现三态（implemented/stub/not_found），并为每个至少给一条证据。

项目	方法A	方法B
答案	高价值 syscall 实现状态： - sys_fork: implemented [kernel/sched/proc.c:fork()] - sys_exec: implemented [kernel/exec.c:exec()] - sys_mmap: implemented [kernel/syscall/sysmem.c:sys_mmap()] - sys_openat: implemented [kernel/syscall/sysfile.c:sys_openat()] - sys_write: implemented [kernel/syscall/sysfile.c:sys_write()] - sys_clone: implemented [kernel/sched/proc.c:clone()] - sys_wait4: implemented [kernel/sched/proc.c:wait4()] - sys_getuid: stub [kernel/syscall/sysproc.c:267-269 仅返回 0] - sys_geteuid: stub [kernel/syscall/syscall.c:233 指向 sys_getuid] - sys_getgid: stub [kernel/syscall/syscall.c:234 指向 sys_getuid]	fork: implemented [kernel/syscall/sysproc.c], exec: implemented [kernel/syscall/sysproc.c], mmap: implemented [kernel/syscall/sysmem.c], open: implemented [kernel/syscall/sysfile.c], write: implemented [kernel/syscall/sysfile.c], kill: implemented [kernel/syscall/syssignal.c], brk: implemented [kernel/syscall/sysmem.c], wait: implemented [kernel/sched/proc.c], exit: implemented [kernel/sched/proc.c], getpid: implemented [kernel/syscall/sysproc.c]
证据	[1] `kernel/sched/proc.c` · fork `int fork(void) { ... }` [2] `kernel/exec.c` · exec `int exec(char *path, char *argv, char *envp) { ... }` [3] `kernel/syscall/sysproc.c` · sys_getuid `uint64 sys_getuid(void) { return 0; }`	[1] `kernel/syscall/sysproc.c` · sys_fork `uint64 sys_fork(void) { return clone(0, NULL); }` [2] `kernel/syscall/sysproc.c` · sys_exec `uint64 sys_exec(void) { ... execve(ecmd->argv[0], ecmd->argv, 0); }` [3] `kernel/syscall/sysmem.c` · sys_mmap `uint64 sys_mmap(void) { ... return do_mmap(start, len, prot, flags, f, off); }` [4] `kernel/syscall/sysfile.c` · sys_openat `uint64 sys_openat(void) { ... fd = fdalloc(f, omode & O_CLOEXEC); }` [5] `kernel/syscall/sysfile.c` · sys_write `uint64 sys_write(void) { ... return filewrite(f, p, n); }`

`Q02_025` · `short_answer`

> 缺页异常与内存特性（CoW/lazy）是否在 trap 中联动？（若存在，说明入口点与调用到内存模块的证据）

项目	方法A	方法B
答案	存在联动。入口点：`kernel/trap/trap.c:handle_excp()` 检测页面异常 → 调用 `kernel/mm/vm.c:handle_page_fault()`。CoW 处理：`vm.c:handle_store_page_fault_cow()` 检测 PTE_COW 标志并复制页面。Lazy 分配：`vm.c:handle_page_fault_lazy()` 为 HEAP/STACK 段按需分配页面。证据：`trap.c:320-330` 异常分发，`vm.c:783-850` 缺页处理完整链路。	是，handle_page_fault() 在 kernel/trap/trap.c 中调用，根据 seg type 调用 handle_page_fault_lazy() 或 handle_page_fault_loadelf() 或 handle_page_fault_mmap()

项目	方法A	方法B
证据	[1] `kernel/trap/trap.c` · handle_excp `case EXCP_STORE_PAGE: return handle_page_fault(1, r_stval());` [2] `kernel/mm/vm.c` · handle_page_fault `int handle_page_fault(int kind, uint64 badaddr) { ... switch (seg->type) { case LOAD: ... case HEAP: case STACK: return handle_page_fault_lazy(...); } }` [3] `kernel/mm/vm.c` · handle_store_page_fault_cow `static int handle_store_page_fault_cow(pte_t ptep) { if (monopolizepage(pa)) { pte |= PTE_W; } else { char copy = (char *)allocpage(); ... } }`	[1] `kernel/trap/trap.c` · handle_excp `if (is_page_fault(scause)) return handle_page_fault(kind, r_stval());` [2] `kernel/mm/vm.c` · handle_page_fault `switch (seg->type) { case LOAD: return handle_page_fault_loadelf(...); case HEAP: case STACK: return handle_page_fault_lazy(...); case MMAP: return handle_page_fault_mmap(...); }` [3] `kernel/mm/vm.c` · handle_store_page_fault_cow `if (monopolizepage(pa)) { pte |= PTE_W; } else { copy = allocpage(); memmove(copy, (char *)pa, PGSIZE); }`

`Q02_026` · `short_answer`

> 与 09 多核交叉一致性：per-CPU trap 栈/时钟初始化顺序与 AP 上线是否一致？（互指证据或写单核不适用）

项目	方法A	方法B
答案	多核一致。`kernel/main.c:main()` 中 hart 0 先初始化 `trapinithart()`，然后通过 `sbi_send_ipi()` 唤醒其他 hart。其他 hart 在 `started == 1` 后也调用 `trapinithart()`。每 CPU 通过 `tp` 寄存器存储 hartid（`main.c:inithartid()`）。时钟初始化在 `trapinithart()` 中通过 `set_next_timeout()` 完成。证据：`main.c:45-75` 多核启动序列，`trap.c:52` 每 hart 陷阱初始化。	多核支持：kernel/main.c 中 hart 0 初始化后通过 sbi_send_ipi() 唤醒其他核，每个核独立调用 trapinithart() 和 floatinithart()
证据	[1] `kernel/main.c` · main `for (int i = 1; i < NCPU; i ++) { sbi_send_ipi(mask, 0); } ... started = 1; ... else { floatinithart(); kvminithart(); trapinithart(); }` [2] `kernel/trap/trap.c` · trapinithart `void trapinithart(void) { w_stvec((uint64)kernelvec); ... set_next_timeout(); }`	[1] `kernel/main.c` · main `if (hartid == 0) { ... for(int i = 1; i < NCPU; i++) { sbi_send_ipi(1 << i, 0); } } else { floatinithart(); kvminithart(); trapinithart(); }` [2] `kernel/sched/proc.c` · mycpu `struct cpu *mycpu(void) { int id = cpuid(); return &cpus[id]; }`

`Q02_027` · `fill_in`

> Syscall 实现全量统计 (Syscall Coverage Analysis)，请按格式填写： - 分发表路径: ___ - 完整实现 ✅ (implemented): ___ 个 - 桩/ENOSYS/return 0 🔸 (stub): ___ 个，代表性例子: ___ - 未注册 ❌ (not_found): ___ 个 - 统计依据（grep 或 outline 方式）: ___ （若无法精确计数，给出区间估计并说明理由）

项目	方法A	方法B
答案	分发表路径：kernel/syscall/syscall.c:194-258 (syscalls[] 数组) 完整实现 ✅ (implemented): 约 55 个 (sys_fork, sys_exec, sys_write, sys_read, sys_openat, sys_mmap, sys_clone, sys_wait4 等有完整逻辑) 桩/ENOSYS/return 0 🔸 (stub): 约 5 个，代表性例子：sys_getuid (仅返回 0), sys_geteuid (指向 sys_getuid), sys_getgid (指向 sys_getuid), sys_getegid (指向 sys_getuid), sys_prlimit64 (仅返回 0) 未注册 ❌ (not_found): 0 个 (所有 SYS_*常量都在 syscalls[] 中有注册，即使是指向桩函数*) *统计依据：grep kernel/syscall/syscall.c 的 syscalls[] 数组，共 68 个条目；逐个检查 sys_*.c 文件中的实现体深度*	分发表路径：kernel/syscall/syscall.c (syscalls[] 数组) 完整实现 ✅ (implemented): 约 50+ 个 桩/ENOSYS/return 0 🔸 (stub): 少数几个，如 sys_prlimit64 return 0 未注册 ❌ (not_found): 约 10-15 个（sysnum.h 中定义但未在 syscalls[] 中注册） 统计依据（grep 或 outline 方式）: 对比 include/sysnum.h 中 SYS_* 定义与 kernel/syscall/syscall.c 中 syscalls[] 数组索引

项目	方法A	方法B
证据	[1] `kernel/syscall/syscall.c` · syscalls[] `static uint64 (*syscalls[])(void) = { [SYS_fork] sys_fork, ... [SYS_msync] sys_msync }; // 共 68 个条目` [2] `kernel/syscall/sysproc.c` · sys_getuid `uint64 sys_getuid(void) { return 0; }`	[1] `include/sysnum.h` · SYS_ *`#define SYS_fork 1 ... #define SYS_msync 227 (约 70+ syscall numbers defined)`* *[2] `kernel/syscall/syscall.c` · syscalls* *`static uint64 (*syscalls[])(void) = { [SYS_fork] sys_fork, ... [SYS_msync] sys_msync };`* [3] `kernel/syscall/sysproc.c` · sys_prlimit64 `uint64 sys_prlimit64(void) { return 0; } // stub implementation`

`Q02_028` · `short_answer`

> README 与 syscall 声称对照：README 中声称兼容/实现了哪些 syscall 或标准？与代码分发表实际是否一致？（无 README 则写「无 README，仅以代码为准」）

项目	方法A	方法B
答案	README.md 未明确列出 syscall 兼容性声称，仅在 Progress 章节列出功能进度（进程管理、文件系统等）。doc/用户使用 - 系统调用.md 提到支持标准 POSIX syscall。代码分发表实际实现了 68 个 syscall，覆盖 fork/exec/wait/read/write/open/close/mmap 等核心功能，与 README 声称的"进程管理"、"文件系统"功能一致。	README.md 提到支持部分 POSIX 接口的系统调用，可以运行静态链接的 ELF 可执行程序；代码中实现了大量 syscall 包括 fork/exec/wait/read/write/open/mmap 等，与 POSIX 兼容
证据	[1] `README.md` · Progress `## Progress - [x] Process management - [x] File system`	[1] `README.md` · Progress `[x] Process management [x] File system [x] User program` [2] `doc/总言.md` · 系统框架 `支持部分 POSIX 接口的系统调用，可以运行静态链接的 ELF 可执行程序`

`Q02_029` · `short_answer`

> `_impl` 命名模式搜索结论：grep `_impl\b|sys_[a-z0-9_]*_impl`，结果是命中了哪些函数（列出），还是「未见该命名模式」？（必须给搜索结论）

项目	方法A	方法B
答案	未见该命名模式。搜索 `_impl\b|sys_[a-z0-9_]*_impl` 在 152 个文件中无匹配结果。xv6-k210 采用直接命名（如 `sys_write`），未使用 `_impl` 后缀分离接口与实现。	未见该命名模式
证据	—	[1] `kernel/syscall/` · syscall_files `Searched kernel/syscall/.c for impl pattern - no matches found. Functions use sys naming convention without _impl suffix`

`Q02_032` · `short_answer`

> 信号发送支持哪些粒度？（搜索 sys_kill/sys_tkill/sys_tgkill；分别是进程级/线程级/进程组级；列出已实现的与其证据）

项目	方法A	方法B
答案	仅支持进程级信号发送。实现了 `kernel/syscall/syssignal.c:sys_kill()` (第 134 行)，通过 `kill(pid, sig)` 向进程发送信号。未发现 sys_tkill (线程级) 和 sys_tgkill (进程组级) 的实现。搜索 sys_tkill 和 sys_tgkill 无匹配结果。	仅实现进程级 sys_kill，不支持线程级 (tkill) 或进程组级 (tgkill)
证据	[1] `kernel/syscall/syssignal.c` · sys_kill `uint64 sys_kill(void) { int pid, sig; argint(0, &pid); argint(1, &sig); return kill(pid, sig); }`	[1] `kernel/syscall/syssignal.c` · sys_kill `uint64 sys_kill(void) { int pid, sig; argint(0, &pid); argint(1, &sig); return kill(pid, sig); }` [2] `include/sysnum.h` · SYS_kill `#define SYS_kill 129 - only kill syscall defined, no tkill/tgkill`

03_mem_mgmt — 内存管理（物理/虚拟/分配器）

`Q03_004` · `short_answer`

> 物理页帧分配器的核心数据结构是什么？（例如 bitmap 数组、buddy free list、slab cache 表、 `struct run` 单链表等；必须引用结构体/字段证据）

项目	方法A	方法B
答案	struct run 单链表 + struct pm_allocator 分桶管理。struct run 包含 next 指针和 npage 字段表示连续页数；struct pm_allocator 包含 spinlock 锁、freelist 链表头和 npage 总页数。系统维护 single 和 multiple 两个分配器实例，分别管理单页和多页分配。	struct run 链表，包含 next 指针和 npage 字段表示连续页数
证据	[1] `kernel/mm/pm.c` · run `struct run { struct run *next; uint64 npage; }` *[2]* `kernel/mm/pm.c` · *pm_allocator* `struct pm_allocator { struct spinlock lock; struct run freelist; uint64 npage; }` [3] `kernel/mm/pm.c` · single `struct pm_allocator single;` 管理单页 [4] `kernel/mm/pm.c` · multiple `struct pm_allocator multiple;` 管理多页	[1] `kernel/mm/pm.c` · run `struct run { struct run *next; uint64 npage; }` - *linked list node where next points to next free run and npage stores number of contiguous pages* *[2]* `kernel/mm/pm.c` · *pm_allocator* `struct pm_allocator { struct spinlock lock; struct run freelist; uint64 npage; }` - freelist points to head of run list

`Q03_005` · `short_answer`

> 物理分配器的并发控制锁粒度是什么？（全局大锁 / per-CPU / 分桶 / 无锁+关中断 / 其他；必须给锁对象类型与持锁范围证据）

项目	方法A	方法B
答案	分桶锁（双锁设计）。single 和 multiple 两个分配器各持有一个独立的 spinlock。alloc/free 操作通过 __enter_sin_cs/__leave_sin_cs 或 __enter_mul_cs/__leave_mul_cs 宏获取对应锁，持锁范围覆盖整个分配/释放操作全程。	全局大锁（global spinlock），使用 struct spinlock 保护整个分配器

项目	方法A	方法B
证据	[1] `kernel/mm/pm.c` · __enter_sin_cs `#define __enter_sin_cs acquire(&single.lock);` [2] `kernel/mm/pm.c` · __enter_mul_cs `#define __enter_mul_cs acquire(&multiple.lock);` [3] `kernel/mm/pm.c` · allocpage_n `__enter_mul_cs ret = __mul_alloc_no_lock(n); __leave_mul_cs`	[1] `kernel/mm/pm.c` · pm_allocator `struct pm_allocator { struct spinlock lock; ... } - single spinlock protects entire allocator` [2] `kernel/mm/pm.c` · __enter_mul_cs `#define __enter_mul_cs acquire(&multiple.lock) - acquires global lock before allocation` [3] `kernel/mm/pm.c` · __leave_mul_cs `#define __leave_mul_cs release(&multiple.lock) - releases global lock after allocation`

`Q03_007` · `short_answer`

> 页表操作 API（walk/map/unmap 或等价）对应的函数名/模块是什么？列出 1-3 个关键入口并给证据。

项目	方法A	方法B
答案	核心 API：walk() 用于页表遍历（kernel/mm/vm.c:211），mappages() 用于建立映射（kernel/mm/vm.c:280），unmappages() 用于解除映射（kernel/mm/vm.c:337）。辅助 API：uvmalloc() 用于用户地址空间增长（kernel/mm/vm.c:417），walkaddr() 用于用户地址验证（kernel/mm/vm.c:227）。	walk(), mappages(), unmappages() 在 kernel/mm/vm.c 中实现
证据	[1] `kernel/mm/vm.c` · walk `pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) 行 211` [2] `kernel/mm/vm.c` · mappages `int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm) 行 280` [3] `kernel/mm/vm.c` · unmappages `void unmappages(pagetable_t pagetable, uint64 va, uint64 npages, int flag) 行 337`	[1] `kernel/mm/vm.c` · walk `pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) - returns PTE address for va, allocates intermediate pages if needed` [2] `kernel/mm/vm.c` · mappages `int mappages(pagetable_t pt, uint64 va, uint64 size, uint64 pa, int perm) - maps virtual address range to physical with permissions` [3] `kernel/mm/vm.c` · unmappages `void unmappages(pagetable_t pt, uint64 va, uint64 npages, int flag) - unmaps pages, optionally frees physical memory`

`Q03_008` · `short_answer`

> 页表修改路径的并发控制是什么？（锁粒度、是否需要关中断、是否使用每进程地址空间锁等；必须引用锁/临界区证据）

项目	方法A	方法B
答案	依赖进程地址空间隔离 + 关中断。页表修改路径（walk/mappages/unmappages）本身无显式每页表锁，但通过以下机制保证安全：(1) 每个进程有独立的 pagetable，用户态页表修改在进程上下文中进行；(2) 内核态页表修改时通过 intr_off() 关中断（如 usertrapret 中）；(3) trap 处理路径中 handle_page_fault 在关中断的 kerneltrap 上下文中执行。	使用进程锁 (struct proc->lk) 和全局锁保护，页表修改通常在持有进程锁时进行，不需要显式关中断

项目	方法A	方法B
证据	[1] `kernel/trap/trap.c` · usertrapret `intr_off()` 在切换页表前关中断 [2] `kernel/mm/vm.c` · uvmcopy `sfence_vma()` 在页表修改后刷新 TLB	[1] `kernel/sched/proc.c` · proc `struct proc { struct spinlock lk; ... pagetable_t pagetable; ... } - each process has lock protecting its resources including pagetable` [2] `kernel/mm/vm.c` · uvmcopy `Called during fork with parent process lock held, copies page table entries with COW support` [3] `kernel/mm/vm.c` · handle_page_fault `Page fault handler allocates pages and calls mappages, called from trap handler context`

`Q03_011` · `short_answer`

> 追踪一条缺页链路：trap/异常入口 → 缺页处理函数（handle_page_fault 或等价）→ 分配页帧 → 建立映射。用 3-5 个关键节点描述并给每节点证据。

项目	方法A	方法B
答案	缺页链路：(1) kerneltrap() [kernel/trap/trap.c:206] 捕获异常 → (2) handle_excp() [kernel/trap/trap.c:323] 识别缺页类型 → (3) handle_page_fault() [kernel/mm/vm.c:1039] 根据 seg 类型分发 → (4) handle_page_fault_lazy() [kernel/mm/vm.c:1002] 调用 uvmalloc() → (5) uvmalloc() [kernel/mm/vm.c:417] 调用 allocpage() 和 mappages() 建立映射 → (6) sfence_vma() 刷新 TLB。	kernelvec.S:kerneltrap → trap.c:handle_excp → vm.c:handle_page_fault → pm.c:_allocpage → vm.c:mappages
证据	[1] `kernel/trap/trap.c` · handle_excp `return handle_page_fault(1, r_stval()) 行 329` [2] `kernel/mm/vm.c` · handle_page_fault_lazy `uvmalloc(p->pagetable, pa, pa + PGSIZE, s->flag) 行 1007` [3] `kernel/mm/vm.c` · uvmalloc `mem = allocpage(); mappages(pagetable, a, PGSIZE, (uint64)mem, perm|PTE_U)`	[1] `kernel/trap/kernelvec.S` · kerneltrap `Assembly trap handler saves registers and calls C kerneltrap() function` [2] `kernel/trap/trap.c` · kerneltrap `void kerneltrap() - calls handle_excp() for page fault exceptions` [3] `kernel/trap/trap.c` · handle_excp `int handle_excp(uint64 scause) - dispatches to handle_page_fault() for load/store faults` [4] `kernel/mm/vm.c` · handle_page_fault `Calls allocpage() or handle_page_fault_lazy() then mappages() to resolve fault` [5] `kernel/mm/pm.c` · _allocpage `uint64 _allocpage(void) - allocates physical page from freelist for fault handler`

`Q03_016` · `tri_state_impl`

> 是否存在 Page Cache（页缓存/文件页缓存）管理？（必须三态）

项目	方法A	方法B
答案	implemented	stub
证据	[1] `kernel/fs/bio.c` · bio.c `Buffer cache 实现，使用 struct buf 缓存磁盘块，LRU 链表管理` [2] `kernel/fs/bio.c` · bcache `static list_node_t *bcache[BCACHE_TABLE_SIZE] 哈希表` [3] `kernel/fs/bio.c` · lru_head `static struct d_list lru_head LRU 链表头`	[1] `kernel/fs/buf.c` · buf `struct buf with LRU list and hash table for block caching, but this is block cache not page cache` [2] `kernel/mm/mmap.c` · mmap_page `struct mmap_page for tracking mapped file pages with ref count, but limited to mmap not general page cache` [3] `kernel/fs` · page_cache_search `No general page cache implementation like Linux page_cache, only buffer cache for disk blocks`

`Q03_017` · `tri_state_impl`

> 是否存在脏页回写 (dirty page writeback) 机制？ (必须三态；若 implemented 需指出同步/异步与触发条件)

项目	方法A	方法B
答案	implemented	stub
证据	[1] `kernel/fs/bio.c` · bwrite `行 199，异步写回：disk_submit() 提交到磁盘驱动队列，不等待完成` [2] `kernel/fs/bio.c` · dirty_buffer_writeback `注释说明：Dirty buffer write back no-block mechanism，异步提交到磁盘驱动`	[1] `kernel/fs/buf.c` · bwrite `void bwrite(struct buf *b) - writes dirty buffer to disk, but this is block-level not page-level` [2] `kernel/mm/mmap.c` · do_msync `int do_msync(uint64 addr, uint64 len, int flags) - syncs mmap'd pages to file, triggered by msync() syscall` [3] `kernel/fs` · writeback_search `No background writeback daemon or periodic dirty page writeback found`

`Q03_019` · `short_answer`

> TLB 刷新指令/函数点是什么？ (RISC-V sfence.vma / AArch64 tlbi / x86 invlpg 等，或仓库中等价的 TLB 刷新封装；必须给证据)

项目	方法A	方法B
答案	sfence_vma() 函数 [include/hal/riscv.h:362]。QEMU 模式下使用 sfence.vma 指令，K210 实机使用 .word 0x10400073（sfence.vm 的机器码）。调用点：uvmcopy() 行 588、handle_store_page_fault_cow() 行 996、handle_page_fault_lazy() 行 1013、do_mmap() 行 773 等。	sfence_vma() 函数封装 RISC-V sfence.vma 指令，在 include/hal/riscv.h 中定义
证据	[1] `include/hal/riscv.h` · sfence_vma `static inline void sfence_vma() { asm volatile(".word 0x10400073"); }` [2] `kernel/mm/vm.c` · uvmcopy `sfence_vma() 行 588`	[1] `include/hal/riscv.h` · sfence_vma `static inline void sfence_vma() { asm volatile("sfence.vma"); } - RISC-V TLB flush instruction` [2] `kernel/mm/vm.c` · sfence_usage `sfence_vma() called after mappages(), unmappages(), and page table switches to flush TLB` [3] `kernel/trap/trap.c` · usertrapret `sfence_vma() called in usertrapret() when switching to user page table`

`Q03_020` · `short_answer`

> 用户指针安全检查机制是什么？（access_ok/verify_area/UserInPtr 等；列出入口点与校验逻辑证据）

项目	方法A	方法B
答案	双重保护机制：(1) 硬件页表权限位：walkaddr() 检查 PTE_V 和 PTE_U 位 [kernel/mm/vm.c:227-243]；(2) 软件段检查：copyin2()/copyout2() 使用 partofseg() 验证地址是否在进程的 struct seg 链表范围内 [kernel/mm/vm.c:768-780]。safememmove() 使用 permit_usr_mem()/protect_usr_mem() 切换 SSTATUS_SUM 位控制用户态访问权限。	使用 rangeinseg() 检查地址是否在进程段内，配合 copyin/copyout 进行安全拷贝
证据	[1] `kernel/mm/vm.c` · walkaddr `检查 PTE_V 和 PTE_U 位，返回物理地址或 NULL` [2] `kernel/mm/vm.c` · copyout2 `struct seg *s = partofseg(p->segment, dstva, dstva + len) 行 768` [3] `kernel/mm/vm.c` · safememmove `permit_usr_mem() 和 protect_usr_mem() 控制 SSTATUS_SUM 位`	[1] `kernel/mm/usrmm.c` · rangeinseg `int rangeinseg(uint64 start, uint64 end) - checks if address range is within process segments` [2] `kernel/mm/vm.c` · copyin2 `int copyin2(char dst, uint64 srcva, uint64 len) - safely copies from user to kernel with segment validation` *[3] `kernel/mm/vm.c` · copyout2* `int copyout2(uint64 dstva, char src, uint64 len) - safely copies from kernel to user with segment validation` [4] `kernel/mm/vm.c` · safememmove `Handles page faults during copy operations with safe escape mechanism via kern_pgfault_escape()`

`Q03_023` · `fill_in`

> 物理内存总量（Physical Memory Size）：_ ***KB/MB；页大小（Page Size）：*** _ bytes；最大进程虚拟地址空间（Virtual Address Space）：____ bits。（必须从代码常量/链接脚本/配置中给出证据；无法确定则写 unknown 并说明已搜路径）

项目	方法A	方法B
答案	物理内存总量：6 MB（PHYSTOP 0x80600000 - KERNBASE 0x80020000 ≈ 6MB）；页大小：4096 bytes（PGSIZE）；最大进程虚拟地址空间：39 bits（Sv39，MAXVA = 1L << (9+9+9+12-1) = 2^38，实际可用 38 位，但 Sv39 支持 39 位虚拟地址）。	物理内存总量：4 MB (PHYSTOP 0x80600000 - KERNBASE 0x80020000)；页大小：4096 bytes；最大进程虚拟地址空间：39 bits (Sv39)
证据	[1] `include/memlayout.h` · PHYSTOP `#define PHYSTOP 0x80600000UL 行 102` [2] `include/memlayout.h` · KERNBASE `#define KERNBASE 0x80020000UL 行 99` [3] `include/hal/riscv.h` · PGSIZE `#define PGSIZE 4096 行 378` [4] `include/hal/riscv.h` · MAXVA `#define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) 行 408`	[1] `include/memlayout.h` · PHYSTOP `#define PHYSTOP 0x80600000UL - end of physical memory` [2] `include/memlayout.h` · KERNBASE `#define KERNBASE 0x80020000UL - kernel base address` [3] `include/hal/riscv.h` · PGSIZE `#define PGSIZE 4096 - page size in bytes` [4] `include/hal/riscv.h` · MAXVA `#define MAXVA (1L << (9 + 9 + 9 + 12 - 1)) - Sv39 39-bit virtual address space` [5] `linker/linker64.ld` · memory_layout `BASE_ADDRESS = 0x80020000 - kernel load address matches KERNBASE`

`Q03_025` · `short_answer`

> 逻辑内存组织 (Logical Memory Organization, Stallings Ch7.1)：进程地址空间中 text/data/heap/stack/mmap 各区域（或等价区间）是否由统一的映射管理结构（VMA/区间表/链表/BTreeMap 等）维护？（如存在请给结构体证据；不存在则写未发现等价结构）

项目	方法A	方法B
答案	是，使用 struct seg 链表维护。struct seg 包含 type（LOAD/TEXT/DATA/BSS/HEAP/MMAP/STACK）、addr（起始地址）、sz（大小）、flag（权限）、next（链表指针）等字段。进程控制块 struct proc 包含 segment 字段指向 seg 链表头。	使用 struct seg 链表统一管理，包含 type 字段区分 LOAD/TEXT/DATA/BSS/HEAP/MMAP/STACK 等段类型
证据	[1] `include/mm/usrmm.h` · seg `struct seg { enum segtype type; int flag; uint64 addr; uint64 sz; struct seg next; ... }` *[2]* `include/sched/proc.h` · *proc* `struct proc 包含 struct seg segment 字段`	[1] `include/mm/usrmm.h` · segtype `enum segtype { NONE, LOAD, TEXT, DATA, BSS, HEAP, MMAP, STACK } - segment types` [2] `include/mm/usrmm.h` · seg `struct seg { enum segtype type; int flag; uint64 addr; uint64 sz; struct seg next; uint64 mmap; ... } - segment descriptor with linked list` *[3]* `kernel/mm/usrmm.c` · *locateseg* `struct seg locateseg(struct seg head, uint64 addr) - finds segment containing address` *[4]* `kernel/sched/proc.h` · *proc* `struct proc { ... struct seg segment; ... } - each process has segment list head`

`Q03_028` · `short_answer`

> 放置策略 (Placement Policy, Stallings Ch8.2)：新的匿名映射或堆区域增长时，系统如何选择虚拟地址区间？（固定起始地址 / mmap_base 向下生长 / 首次适配 / 最佳适配 等；必须给实现证据或写未发现等价策略）

项目	方法A	方法B
答案	首次适配（first-fit）策略。lookup_segment() 在进程的 struct seg 链表中顺序查找第一个足够大的空闲间隙。对于 mmap，lookup_fixed_segment() 支持 MAP_FIXED 固定地址映射，否则使用 lookup_segment() 查找合适位置。	使用首次适配 (first-fit) 策略，从 VUMMAP (0x70000000) 开始向上查找空闲区域
证据	[1] `kernel/mm/mmap.c` · lookup_segment `遍历 seg 链表查找空闲间隙` [2] `kernel/mm/mmap.c` · lookup_fixed_segment `处理 MAP_FIXED 标志，删除现有映射`	[1] `kernel/mm/mmap.c` · lookup_segment `Searches from VUMMAP upward, finds first gap between existing segments for new mapping` [2] `include/memlayout.h` · VUMMAP `#define VUMMAP 0x70000000 - base address for mmap region` [3] `kernel/mm/mmap.c` · lookup_fixed_segment `For MAP_FIXED, validates requested address and splits existing segments if needed`

`Q03_030` · `short_answer`

> 内存主链路（必须给出，尽量以 Mermaid graph TD 表达）：从确认的最强内存入口（缺页处理入口/mmap 入口/brk 入口/等价入口）出发，追踪到页表操作核心点或物理页分配核心点，写出 3-6 个关键节点。节点格式：FuncName [path:line]。若链路未被源码证据完全闭合，标注候选主链路而非确认的主链路。只画一条主链，不要并列展开多条支线。

项目	方法A	方法B
答案	graph TD\n kerneltrap[kerneltrap kernel/trap/trap.c:206] --> handle_excp[handle_excp kernel/trap/trap.c:323]\n handle_excp --> handle_page_fault[handle_page_fault kernel/mm/vm.c:1039]\n handle_page_fault --> handle_page_fault_lazy[handle_page_fault_lazy kernel/mm/vm.c:1002]\n handle_page_fault_lazy --> uvmalloc[uvmalloc kernel/mm/vm.c:417]\n uvmalloc --> mappages[mappages kernel/mm/vm.c:280]\n mappages --> allocpage[allocpage kernel/mm/pm.c:233]	graph TD\nhandle_page_fault[kernel/mm/vm.c:handle_page_fault] --> handle_page_fault_lazy[kernel/mm/vm.c:handle_page_fault_lazy]\nhandle_page_fault_lazy --> _allocpage[kernel/mm/pm.c:_allocpage]\n_allocpage --> mappages[kernel/mm/vm.c:mappages]\nmappages --> walk[kernel/mm/vm.c:walk]
证据	[1] `kernel/trap/trap.c` · handle_excp `调用 handle_page_fault(1, r_stval()) 行 329` [2] `kernel/mm/vm.c` · handle_page_fault_lazy `调用 uvmalloc() 行 1007` [3] `kernel/mm/vm.c` · uvmalloc `调用 allocpage() 和 mappages() 行 426-434`	[1] `kernel/mm/vm.c` · handle_page_fault `Entry point for page fault handling, dispatches to specific handlers based on segment type` [2] `kernel/mm/vm.c` · handle_page_fault_lazy `Allocates physical page for lazy segments, calls uvmalloc which calls mappages` [3] `kernel/mm/pm.c` · _allocpage `Core physical page allocator, removes page from freelist` [4] `kernel/mm/vm.c` · mappages `Creates page table entries mapping virtual to physical addresses` [5] `kernel/mm/vm.c` · walk `Walks page table hierarchy to find or create PTE for virtual address`

`Q03_034` · `tri_state_impl`

> 是否存在清理策略 (Cleaning Policy, Stallings Ch8.2)？ （即脏页预先后台写回，而非仅在置换时才写回；搜索 background writeback / kswapd / cleaner_thread 或等价；必须三态；若 not_found 需列出已搜关键字）

项目	方法A	方法B
答案	implemented	not_found
证据	[1] `kernel/fs/bio.c` · bwrite `异步写回机制：disk_submit() 提交到磁盘驱动队列，不等待完成 行 199` [2] `kernel/fs/bio.c` · dirty_writeback `注释：Dirty buffer write back no-block mechanism，异步提交`	[1] `kernel` · writeback_search `Searched for background_writeback, kswapd, cleaner_thread, pdflush keywords - none found` [2] `kernel/fs/buf.c` · bsync `void bsync(void) - synchronous buffer write, not background writeback daemon` [3] `kernel/mm/mmap.c` · do_msync `Triggered by msync() syscall, not automatic background cleaning`

04_process_smp — 进程/线程/调度与多核

`Q04_001` · `short_answer`

> 执行实体 (Execution Entity) 抽象是什么？ 请按以下格式作答（每项必须有代码证据）： - 顶层类型名: ***(如 Process / Task / Thread / TaskControlBlock) - 结构体路径: ___ - 关键字段（至少列 3 个）: Context***=, State=***, PID***=, TrapFrame= ***- 是否区分 PCB 与 TCB:*** （是 / 否 / 待核实）

项目	方法A	方法B
答案	顶层类型名: struct proc (Process Control Block, PCB) 结构体路径: include/sched/proc.h:51-93 关键字段: Context=context (struct context, 68-80 行), State=state (enum procstate, 62 行), PID=pid (int, 54 行), TrapFrame=trapframe (struct trapframe*, 85 行) 是否区分 PCB 与 TCB: 否 (xv6-k210 仅使用统一的 struct proc 作为执行实体，无独立线程控制块)	顶层类型名：proc (Process) 结构体路径：include/sched/proc.h 关键字段：Context=context (struct context), State=state (enum procstate), PID=pid (int), TrapFrame=trapframe (struct trapframe *) 是否区分 PCB 与 TCB：否
证据	[1] `include/sched/proc.h` · proc `struct proc { int xstate; int pid; enum procstate state; struct context context; struct trapframe *trapframe; ... }` [2] `include/sched/proc.h` · context `struct context { uint64 ra; uint64 sp; uint64 s0-s11; };`	[1] `include/sched/proc.h` · proc `struct proc { int xstate; int pid; ... enum procstate state; ... struct context context; struct trapframe *trapframe; ... }` [2] `include/sched/proc.h` · procstate `enum procstate { RUNNABLE, RUNNING, SLEEPING, ZOMBIE }`

`Q04_002` · `short_answer`

> 任务/进程的生命周期状态机有哪些状态与流转点？（Ready/Running/Blocked/Exited 等；需状态枚举/字段证据）

项目	方法A	方法B
答案	状态枚举 (include/sched/proc.h:38-42): RUNNABLE(就绪), RUNNING(运行), SLEEPING(阻塞/睡眠), ZOMBIE(僵尸) 状态流转点: - RUNNABLE→RUNNING: scheduler() 中 __get_runnable_no_lock() 选中后设置 state=RUNNING (kernel/sched/proc.c:681 行) - RUNNING→RUNNABLE: yield() 或 proc_tick() 超时，调用 __insert_runnable() (kernel/sched/proc.c:627 行/765 行) - RUNNING→SLEEPING: sleep() 调用 __insert_sleep() (kernel/sched/proc.c:595 行) - SLEEPING→RUNNABLE: wakeup() 调用 __insert_runnable(PRIORITY_IRQ) (kernel/sched/proc.c:379 行) - RUNNING→ZOMBIE: exit() 设置 state=ZOMBIE 并 __remove() (kernel/sched/proc.c:447 行) - ZOMBIE→释放: wait4() 找到 ZOMBIE 子进程后调用 freeproc() (kernel/sched/proc.c:513 行)	状态：RUNNABLE (可运行), RUNNING (运行中), SLEEPING (睡眠), ZOMBIE (僵尸) 流转：allocproc() 创建 RUNNABLE -> scheduler() 切换 RUNNING -> sleep() 进入 SLEEPING -> wakeup() 返回 RUNNABLE -> exit() 进入 ZOMBIE -> wait4() 回收
证据	[1] `include/sched/proc.h` · procstate `enum procstate { RUNNABLE, RUNNING, SLEEPING, ZOMBIE };` [2] `kernel/sched/proc.c` · scheduler `tmp->state = RUNNING;` [3] `kernel/sched/proc.c` · exit `p->state = ZOMBIE; __remove(p);`	[1] `include/sched/proc.h` · procstate `enum procstate { RUNNABLE, RUNNING, SLEEPING, ZOMBIE }` [2] `kernel/sched/proc.c` · allocproc `__insert_runnable(PRIORITY_NORMAL, np); // set state to RUNNABLE` [3] `kernel/sched/proc.c` · exit `p->state = ZOMBIE; __remove(p);` [4] `kernel/sched/proc.c` · sleep `__insert_sleep(p); // set state to SLEEPING`

`Q04_004` · `short_answer`

> 上下文切换保存/恢复了哪些寄存器集合？（例如 RISC-V s0-s11；必须引用汇编/结构体证据）

项目	方法A	方法B
答案	保存/恢复的寄存器 (kernel/sched/swtch.S:7-30 行): - ra (返回地址) - sp (栈指针) - s0-s11 (callee-saved 寄存器，共 12 个) 总计 14 个寄存器，每个 8 字节，共 112 字节。 对应 struct context 定义 (include/sched/proc.h:17-30 行): ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11	保存的寄存器：ra, sp, s0-s11 (callee-saved registers) 不保存：t0-t6, a0-a7 (caller-saved, saved in trapframe)
证据	[1] `kernel/sched/swtch.S` · swtch `sd ra, 0(a0); sd sp, 8(a0); sd s0, 16(a0); ... sd s11, 104(a0); ld ra, 0(a1); ... ret` [2] `include/sched/proc.h` · context `struct context { uint64 ra; uint64 sp; uint64 s0; ... uint64 s11; };`	[1] `kernel/sched/swtch.S` · swtch `sd ra, 0(a0) sd sp, 8(a0) sd s0, 16(a0) sd s1, 24(a0) sd s2, 32(a0) sd s3, 40(a0) sd s4, 48(a0) sd s5, 56(a0) sd s6, 64(a0) sd s7, 72(a0) sd s8, 80(a0) sd s9, 88(a0) sd s10, 96(a0) sd s11, 104(a0)` [2] `include/sched/proc.h` · context `struct context { uint64 ra; uint64 sp; uint64 s0; uint64 s1; ... uint64 s11; }`

`Q04_005` · `short_answer`

> 调度算法 (Scheduling Algorithm) 属于哪类？ 请按格式作答： - 算法名称: ***(必须是以下之一：FCFS / Round-Robin (RR) / Stride/Proportional-Share / MLFQ / CFS / Priority / 其他） - 代码证据（关键字段/函数）：___ - RR: timeslice/slice 字段位置=*** - Stride: stride 字段与比较逻辑位置= ***- MLFQ: 多级队列 VecDeque/数组层级证据=*** - Priority: priority 字段参与 pick_next 排序证据=___

项目	方法A	方法B
答案	算法名称: Priority (多级优先级调度 + 时间片超时降级) 代码证据: - 优先级定义: kernel/sched/proc.c:239-243 行定义 PRIORITY_TIMEOUT(0), PRIORITY_IRQ(1), PRIORITY_NORMAL(2) - 优先级队列: struct proc *proc_runnable[PRIORITY_NUMBER] (kernel/sched/proc.c:244 行) - 时间片字段: struct proc 中 int timer (include/sched/proc.h:61 行) - 超时降级: proc_tick() 中 timer 递减至 0 时从 PRIORITY_IRQ/NORMAL 降级到 PRIORITY_TIMEOUT (kernel/sched/proc.c:763-767 行) - 调度选择: __get_runnable_no_lock() 按优先级顺序遍历 proc_runnable[0..2] (kernel/sched/proc.c:543-554 行)	算法名称：Priority 代码证据（关键字段/函数）： - Priority: priority 字段参与 pick_next 排序证据 =proc_runnable[PRIORITY_NUMBER] 三级优先级队列 (PRIORITY_IRQ, PRIORITY_TIMEOUT, PRIORITY_NORMAL)
证据	[1] `kernel/sched/proc.c` · PRIORITY_NUMBER `#define PRIORITY_TIMEOUT 0; #define PRIORITY_IRQ 1; #define PRIORITY_NORMAL 2` [2] `kernel/sched/proc.c` · __get_runnable_no_lock `for (int i = 0; i < PRIORITY_NUMBER; i ++) { tmp = proc_runnable[i]; ... }` [3] `kernel/sched/proc.c` · proc_tick `p->timer = p->timer - 1; if (0 == p->timer) { __remove(p); __insert_runnable(PRIORITY_TIMEOUT, p); }`	[1] `include/sched/proc.h` · PRIORITY_NUMBER `#define PRIORITY_NUMBER 3` [2] `kernel/sched/proc.c` · proc_runnable `struct proc proc_runnable[PRIORITY_NUMBER];` *[3] `kernel/sched/proc.c` · __get_runnable_no_lock* *`for (int i = 0; i < PRIORITY_NUMBER; i++) { tmp = proc_runnable[i]; while (NULL != tmp) { if (RUNNABLE == tmp->state) { return (struct proc )tmp; } } }`*

`Q04_006` · `short_answer`

> 调度器 (Scheduler)核心入口/关键函数有哪些？ （schedule/pick_next 等；给 1-3 个入口与证据）

项目	方法A	方法B
答案	核心入口函数: 1. scheduler() - 主调度循环 (kernel/sched/proc.c:671-711 行): 无限循环调用 __get_runnable_no_lock() 选进程，swtch() 切换上下文 2. __get_runnable_no_lock() - 进程选择 (kernel/sched/proc.c:543-556 行): 按优先级遍历 proc_runnable 队列 3. sched() - 触发切换 (kernel/sched/proc.c:714-749 行): 保存当前 context，swtch 到 cpu->context	核心入口：scheduler() (kernel/sched/proc.c) 关键函数：__get_runnable_no_lock() (选择进程), swtch() (上下文切换), yield() (主动让出), sleep()/wakeup() (阻塞/唤醒)
证据	[1] `kernel/sched/proc.c` · scheduler `void scheduler(void) { while (1) { tmp = __get_runnable_no_lock(); ... swtch(&c->context, &tmp->context); } }` [2] `kernel/sched/proc.c` · __get_runnable_no_lock `static struct proc *__get_runnable_no_lock(void) { for (int i = 0; i < PRIORITY_NUMBER; i ++) { ... } }` [3] `kernel/sched/proc.c` · sched `void sched(void) { ... swtch(&p->context, &mycpu()->context); }`	[1] `kernel/sched/proc.c` · scheduler `void scheduler(void) { ... while(1) { ... tmp = __get_runnable_no_lock(); ... swtch(&c->context, &tmp->context); } }` [2] `kernel/sched/proc.c` · yield `int yield(void) { ... __remove(p); ... __insert_runnable(PRIORITY_NORMAL, p); sched(); }`

`Q04_008` · `short_answer`

> fork/clone 是否复制地址空间与文件表？（必须给复制路径证据；若 stub 需说明形态）

项目	方法A	方法B
答案	是，完整复制: - 地址空间复制: kernel/sched/proc.c:303 行 np->segment = copysegs(p->pagetable, p->segment, np->pagetable) - 文件表复制: kernel/sched/proc.c:321 行 copyfdtable(&p->fds, &np->fds) - 当前目录复制: kernel/sched/proc.c:324 行 np->cwd = idup(p->cwd) - 信号处理复制: kernel/sched/proc.c:310 行 sigaction_copy(&np->sig_act, p->sig_act)	是。地址空间通过 copysegs() 复制页表和段描述符，使用 COW 机制。文件表通过 copyfdtable() 复制文件描述符表，每个打开文件引用计数增加。
证据	[1] `kernel/sched/proc.c` · clone `np->segment = copysegs(p->pagetable, p->segment, np->pagetable); if (copyfdtable(&p->fds, &np->fds) < 0) ...`	[1] `kernel/sched/proc.c` · clone `np->segment = copysegs(p->pagetable, p->segment, np->pagetable); ... if (copyfdtable(&p->fds, &np->fds) < 0) { ... }` [2] `kernel/fs/file.c` · copyfdtable `int copyfdtable(struct fdtable *fdt1, struct fdtable fdt2) { ... fd->arr[i] = filedup(fdt1->arr[i]); ... }` [3] `kernel/mm/vm.c` · uvmcopy `int uvmcopy(pagetable_t old, pagetable_t new, uint64 start, uint64 end, int cow) { ... if (cow && (pte & PTE_W)) { pte = (*pte|PTE_COW) & ~PTE_W; } ... }`

`Q04_012` · `short_answer`

> PID 分配器实现是什么？（自增/bitmap/空闲栈复用/只分配不回收；必须给证据）

项目	方法A	方法B
答案	实现方式: 单调自增 (只分配不回收) 证据: kernel/sched/proc.c:229 行 p->pid = __pid++; 其中 __pid 是全局静态变量 (kernel/sched/proc.c:27 行) PID 哈希表: pid_hash[HASH_SIZE] 用于快速查找 (kernel/sched/proc.c:28-29 行) 无回收机制: 未发现 free_pid 或 release_pid 函数，PID 单调递增不复用	自增分配器。使用全局变量__pid 进行自增，通过 hash_insert_no_lock() 将进程插入 PID 哈希表。
证据	[1] `kernel/sched/proc.c` · __pid `int __pid;` [2] `kernel/sched/proc.c` · allocproc `p->pid = __pid ++; hash_insert_no_lock(p);`	[1] `kernel/sched/proc.c` · __pid `int __pid;` [2] `kernel/sched/proc.c` · allocproc `__enter_hash_cs p->pid = __pid ++; hash_insert_no_lock(p); __leave_hash_cs`

`Q04_013` · `short_answer`

> 父子进程树如何存储？（children Vec/链表/parent+sibling 指针；必须给结构体字段证据）

项目	方法A	方法B
答案	存储方式: 链表 (child + sibling_next/sibling_pprev 指针) 结构体字段 (include/sched/proc.h:74-77 行): - struct proc *child: 指向第一个子进程* - *struct proc* parent: 指向父进程 - struct proc *sibling_next: 指向下一个兄弟进程* - *struct proc* *sibling_pprev: 指向前一个兄弟的 sibling_next 字段 遍历方式: 从 parent->child 开始，沿 sibling_next 遍历所有子进程 (kernel/sched/proc.c:485-517 行)	使用 children 链表 (child 指针) 和 sibling 链表 (sibling_next/sibling_pprev 指针)。parent 指针指向父进程。
证据	[1] `include/sched/proc.h` · proc `struct proc *child; struct proc* parent; struct proc *sibling_next; struct proc *sibling_pprev;` [2] `kernel/sched/proc.c` · wait4 `np = p->child; while (NULL != np) { ... np = np->sibling_next; }`	[1] `include/sched/proc.h` · proc `struct proc *child; struct proc* parent; struct proc *sibling_next; struct proc *sibling_pprev;` [2] `kernel/sched/proc.c` · clone `np->parent = p; np->sibling_pprev = &(p->child); np->sibling_next = p->child; if (NULL != p->child) { p->child->sibling_pprev = &(np->sibling_next); } p->child = np;`

`Q04_015` · `short_answer`

> 与 09 多核的交叉一致性：是否存在每核队列/任务迁移/IPI resched？（需与第 9 章互指证据或写不适用）

项目	方法A	方法B
答案	每核运行队列: 否 (全局共享 proc_runnable[PRIORITY_NUMBER] 队列，无 per-CPU 队列) 任务迁移: 不适用 (单全局队列，无需迁移) IPI resched: 是 (kernel/sched/proc.c:386-389 行 wakeup() 中 sbi_send_ipi() 唤醒另一核) 多核调度: 全局 proc_lock 保护，两核竞争同一队列 (kernel/sched/proc.c:245 行)	存在每核运行队列 (proc_runnable[PRIORITY_NUMBER])，通过 IPI 唤醒其他 CPU。任务迁移通过 scheduler() 在各核独立调度实现。IPI 通过 sbi_send_ipi() 实现。

项目	方法A	方法B
证据	[1] `kernel/sched/proc.c` · proc_runnable `struct proc *proc_runnable[PRIORITY_NUMBER];` [2] `kernel/sched/proc.c` · wakeup `if (flag && avail) { sbi_send_ipi(1 << id, 0); }`	[1] `kernel/sched/proc.c` · proc_runnable `struct proc *proc_runnable[PRIORITY_NUMBER];` [2] `kernel/sched/proc.c` · wakeup `int id = 0 == cpuid() ? 1 : 0; int avail = NULL == cpus[id].proc; __leave_proc_cs if (flag && avail) { sbi_send_ipi(1 << id, 0); }` [3] `include/sbi.h` · sbi_send_ipi `static inline struct sbiret sbi_send_ipi(unsigned long hart_mask, unsigned long hart_mask_base)`

`Q04_016` · `short_answer`

> exit() 资源回收路径：调用链是什么？是否真正回收地址空间/文件表/通知父进程？（必须给调用链证据；桩则说明）

项目	方法A	方法B
答案	调用链 (kernel/sched/proc.c:413-456 行): 1. delsegs(p->pagetable, p->segment) - 删除用户段 2. uvmfree(p->pagetable) - 释放页表 3. dropfdtable(&p->fds) - 关闭文件描述符 4. iput(p->cwd) / iput(p->elf) - 释放 inode 5. 子进程重父: 将所有子进程挂载到 __initproc 6. 设置 ZOMBIE 状态: p->state = ZOMBIE; __remove(p) 7. 唤醒父进程: __wakeup_no_lock(p->parent) 8. 调用 sched() 切换到调度器 9. 父进程 wait4() 中 freeproc() 最终释放 PCB	调用链：exit() -> delsegs() (回收段) -> uvmfree() (回收页表) -> dropfdtable() (回收文件表) -> iput() (回收 inode) -> __wakeup_no_lock(parent) (通知父进程)。真正回收所有资源。
证据	[1] `kernel/sched/proc.c` · exit `delsegs(p->pagetable, p->segment); uvmfree(p->pagetable); dropfdtable(&p->fds); iput(p->cwd); p->state = ZOMBIE; sched();`	[1] `kernel/sched/proc.c` · exit `delsegs(p->pagetable, p->segment); p->segment = NULL; uvmfree(p->pagetable); dropfdtable(&p->fds); iput(p->cwd); iput(p->elf); __wakeup_no_lock(p->parent);` [2] `kernel/sched/proc.c` · freeproc `proc_freepagetable(p->pagetable, p->segment); kfree(p->trapframe); freepage(p->kstack);`

`Q04_018` · `tri_state_impl`

> 是否实现 POSIX 资源限制（rlimit/RLIMIT/getrlimit/setrlimit）？（必须三态；若 implemented 需说明支持的资源类型数量及软/硬限制机制）

项目	方法A	方法B
答案	stub	not_found
证据	[1] `kernel/syscall/sysproc.c` · sys_prlimit64 `uint64 sys_prlimit64(void) { // for now it's not very necessary to implement this syscall // may be implemented later return 0; }`	[1] `include/sysnum.h` · rlimit 在 `include/sysnum.h` 中未找到 `SYS_getrlimit, SYS_setrlimit` 等系统调用号 [2] `kernel/syscall/` · rlimit 在 `kernel/syscall/` 目录中未找到 `rlimit` 相关实现

`Q04_023` · `multi_choice`

> 调度器是否追踪/优化以下哪些性能指标 (Scheduling Criteria, Stallings Ch9)？（多选；未发现则留空并在 notes 写 not_found）

项目	方法A	方法B
答案	["CPU 利用率 (CPU Utilization)", "周转时间 (Turnaround Time)", "等待时间 (Waiting Time)", "响应时间 (Response Time)"]	["F. 未发现调度性能统计"]
证据	[1] `include/sched/proc.h` · tms `struct tms { uint64 utime; uint64 stime; uint64 cutime; uint64 cstime; };` [2] `include/sched/proc.h` · proc `int64 vswtch; int64 ivswtch;` [3] `kernel/syscall/sysproc.c` · sys_getrusage `r.ru_nvcsw = p->vswtch; r.ru_nivcsw = p->ivswtch;`	[1] `include/sched/proc.h` · proc `struct proc { ... int64 vswtch; int64 ivswtch; ... } // 仅有上下文切换计数，无完整性能统计` [2] `kernel/sched/proc.c` · throughput|turnaround|waiting|response `未找到吞吐量、周转时间、等待时间、响应时间等性能指标的计算或优化代码`

`Q04_031` · `short_answer`

> 若存在 IPI：发送与处理路径分别在哪些函数/文件？（给关键入口与证据）

项目	方法A	方法B
答案	IPI 发送路径: - sbi_send_ipi() 调用 (kernel/sched/proc.c:388 行 wakeup 函数; kernel/main.c:69 行 main 函数) - SBI 实现位于 bootloader/SBI/rustsbi-k210 (bootloader/SBI/rustsbi-k210/src/main.rs:161-166 行 send_ipi_many) IPI 处理路径: - 通过 trap 机制处理，hart 从 while(started==0) 循环退出后继续初始化 (kernel/main.c:75-82 行) - 无专用 ipi_handler 函数，IPI 仅用于唤醒 secondary CPU	发送路径：sbi_send_ipi() (include/sbi.h) -> wakeup() (kernel/sched/proc.c) 处理路径：handle_intr() 检测 INTR_SOFTWARE (kernel/trap/trap.c) -> sbi_clear_ipi() 清除中断
证据	[1] `kernel/sched/proc.c` · wakeup `sbi_send_ipi(1 << id, 0);` [2] `kernel/main.c` · main `for (int i = 1; i < NCPU; i ++) { sbi_send_ipi(mask, 0); }`	[1] `include/sbi.h` · sbi_send_ipi `SBI_CALL_2(IPI_EID, IPI_SEND_IPI, hart_mask, hart_mask_base)` [2] `kernel/trap/trap.c` · handle_intr `else if (INTR_SOFTWARE == scause) { sbi_clear_ipi(); return 0; }`

`Q04_033` · `short_answer`

> per-CPU 的实现方式是什么？（例如 TLS/tp 寄存器/gsbase/数组索引 hartid；需证据）

项目	方法A	方法B
答案	实现方式: 数组索引 + hartid - 全局数组: struct cpu cpus[NCPU] (kernel/sched/proc.c:93 行) - hartid 获取: cpuid() 函数读取当前 hart ID - 访问方式: mycpu() 返回 &cpus[cpuid()] (kernel/sched/proc.c:96-99 行) - tp 寄存器初始化: kernel/main.c:26 行 inithartid() 中 mv tp, hartid struct cpu 定义未在 proc.h 中显式给出，但通过 mycpu()->proc 访问当前进程 (kernel/sched/proc.c:100 行)	通过 tp 寄存器存储 hartid，使用 cpus[cpuid()] 数组索引访问 per-CPU 数据。cpuid() 通过 r_tp() 读取 tp 寄存器获取当前 CPU ID。

项目	方法A	方法B
证据	[1] `kernel/sched/proc.c` · mycpu `struct cpu *mycpu(void) { int id = cpuid(); return &cpus[id]; }` [2] `kernel/main.c` · inithartid `asm volatile("mv tp, %0" : : "r" (hartid & 0x1));`	[1] `include/hal/riscv.h` · r_tp `static inline uint64 r_tp() { uint64 x; asm volatile("mv %0, tp" : "=r" (x) ); return x; }` [2] `include/sched/proc.h` · cpuid `static inline int cpuid(void) { return r_tp(); }` [3] `kernel/sched/proc.c` · mycpu `struct cpu *mycpu(void) { int id = cpuid(); return &cpus[id]; }`

`Q04_036` · `short_answer`

> 与 03/04/05/08 章的交叉一致性 (Cross-Chapter Consistency)，按以下四项分别作答（每项须给证据路径或写「单核不适用」）： - 03 TLB: 多核页表修改后 TLB 刷新策略= ***- 04 调度: 每核运行队列/负载均衡/IPI resched=*** - 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序= ***- 08 锁: SpinLock 关中断行为在多核下是否安全=***

项目	方法A	方法B
答案	03 TLB: 多核页表修改后 TLB 刷新策略=未发现 TLB shootdown 实现，仅单核 sfence_vma() (kernel/sched/proc.c:685/688 行 scheduler 中) 04 调度: 每核运行队列/负载均衡/IPI resched=全局共享队列 proc_runnable[]，无 per-CPU 队列；IPI 仅用于 wakeup 唤醒 (kernel/sched/proc.c:388 行) 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=hart0 先 trapinithart() 再唤醒 hart1，hart1 后 trapinithart() (kernel/main.c:47/79 行) 08 锁: SpinLock 关中断行为在多核下是否安全=需检查 spinlock 实现 (见 Q04_037)	03 TLB: 多核页表修改后 TLB 刷新策略=sfence_vma() 在 w_satp() 后调用，但无跨核 TLB 刷新 04 调度：每核运行队列/负载均衡/IPI resched=每核独立 proc_runnable 队列，通过 IPI 唤醒，无负载均衡 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=trapinithart() 在每个 hart 上调用，AP 通过 IPI 唤醒后执行 08 锁：SpinLock 关中断行为在多核下是否安全=acquire() 调用 push_off() 关中断，多核下安全
证据	[1] `kernel/sched/proc.c` · scheduler `w_satp(MAKE_SATP(tmp->pagetable)); sfence_vma();` [2] `kernel/main.c` · main `trapinithart(); ... for (int i = 1; i < NCPU; i ++) { sbi_send_ipi(mask, 0); }`	[1] `kernel/mm/vm.c` · mappages `调用 sfence_vma() 但仅刷新本地 TLB` [2] `kernel/sched/proc.c` · wakeup `通过 sbi_send_ipi() 唤醒其他 CPU` [3] `kernel/trap/trap.c` · trapinithart `void trapinithart(void) { w_stvec((uint64)kernelvec); ... }` [4] `kernel/sync/spinlock.c` · acquire `push_off(); // disable interrupts to avoid deadlock.`

`Q04_038` · `short_answer`

> NCPU/MAXCPU（或等价宏）与链接脚本中的每 hart 栈/入口布局是否对应？（搜索 _max_hart_id 等；给宏定义与链接脚本对应证据，或写未发现）

项目	方法A	方法B
答案	NCPU 定义: include/param.h:5 行 #define NCPU 2 链接脚本: bootloader/SBI/rustsbi-k210/link-k210.ld:7 行 _max_hart_id = 1 (支持 2 核，hart0+hart1) 对应关系: NCPU=2 与 _max_hart_id=1 一致 (hart 编号 0-1) 每 hart 栈布局: kernel/main.c:88-92 行 shrink boot stack 中 kstack = boot_stack + hartid * 4 * PGSIZE	NCPU=2 (include/param.h)。链接脚本未显式定义每 hart 栈，但 kernel/entry.S 中通过 add t0, a0, 1; slli t0, t0, 14 计算每核栈偏移 (4096*4 字节)。

项目	方法A	方法B
证据	[1] include/param.h · NCPU #define NCPU 2 // maximum number of CPUs [2] bootloader/SBI/rustsbi-k210/link-k210.ld · _max_hart_id _max_hart_id = 1; [3] kernel/main.c · main uint64 kstack = (uint64)boot_stack + hartid * 4 * PGSIZE;	[1] include/param.h · NCPU #define NCPU 2 // maximum number of CPUs [2] kernel/entry.S · _entry add t0, a0, 1 slli t0, t0, 14 la sp, boot_stack add sp, sp, t0 [3] kernel/entry.S · boot_stack boot_stack: .space 4096 * 4 * 2

05_fs_drivers — 文件系统与设备 I/O

Q05_001 · short_answer

> VFS 抽象层 (Virtual File System, VFS)接口是什么形态？（Rust trait / C op 表；必须给接口定义证据）

项目	方法A	方法B
答案	C 语言函数指针结构体（op 表）形态。定义于 include/fs/fs.h:44-78，包含 struct fs_op（块设备操作）、struct inode_op（inode 操作）、struct dentry_op（目录项操作）、struct file_op（文件操作）四个操作表，每个表包含一组函数指针如 alloc_inode、lookup、read、write 等。	C op 表（函数指针结构体）。在 include/fs/fs.h 中定义了 struct fs_op、struct inode_op、struct file_op、struct dentry_op，每个包含一组函数指针作为操作接口。
证据	[1] include/fs/fs.h · fs_op struct fs_op { struct inode (alloc_inode)(struct superblock sb); ... } [2] include/fs/fs.h · inode_op struct inode_op { struct inode (create)(struct inode ip, char name, int mode); ... } [3] include/fs/fs.h · file_op struct file_op { int (read)(struct inode *ip, int usr, uint64 dst, uint off, uint n); ... }	[1] include/fs/fs.h · fs_op struct fs_op { struct inode (alloc_inode)(struct superblock sb); void (destroy_inode)(struct inode ip); int (write)(struct superblock sb, int usr, char src, uint64 blockno, uint64 off, uint64 len); int (read)(struct superblock sb, int usr, char dst, uint64 blockno, uint64 off, uint64 len); ... }; [2] include/fs/fs.h · inode_op struct inode_op { struct inode (create)(struct inode ip, char name, int mode); struct inode (lookup)(struct inode dir, char filename, uint poff); int (truncate)(struct inode ip); int (unlink)(struct inode ip); ... };

Q05_003 · short_answer

> 若支持 FAT32/Ext4：它是自研还是第三方库/crate？（必须引用 Cargo.toml/Cargo.lock 或 Makefile 引入证据）

项目	方法A	方法B
答案	自研实现。FAT32 后端代码位于 kernel/fs/fat32/ 目录，包含 fat32.c（589 行）、fat32.h、fat_cache.c 等文件，直接编译进内核。Makefile（Makefile:1-80）显示为纯 C 项目，无外部 FS 库依赖。	自研实现。FAT32 文件系统在 kernel/fs/fat32/ 目录下自主实现，包含 cluster.c、dirent.c、fat.c、fat32.c 等源文件，无第三方库依赖。

项目	方法A	方法B
证据	[1] `kernel/fs/fat32/fat32.c` · fat32.c `FAT32 自研实现，包含 fat32_init, fat_read_file, fat_write_file 等` [2] `Makefile` · platform config `platform := k210, 纯 C 编译，无外部 FS crate`	[1] `Makefile` · SRC `SRC += $K/fs/fat32/cluster.c $K/fs/fat32/dirent.c $K/fs/fat32/fat.c $K/fs/fat32/fat32.c` [2] `kernel/fs/fat32/fat32.h` · fat32_header `FAT32 file system implementation header with struct fat32_sb, struct fat32_entry definitions`

`Q05_004` · `short_answer`

> 文件打开路径：文件打开入口（sys_open 或等价）→ VFS 层 → 具体 FS open。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	文件打开调用链：`sys_openat`（`kernel/syscall/sysfile.c:233`）→ `nameifrom` / `namei`（`kernel/fs/fs.c:437`）→ `lookup_path`（`kernel/fs/fs.c:352`）→ `dirlookup`（`kernel/fs/fs.c:253`）→ 具体 FS `fat_lookup_dir`（通过 `ip->op->lookup` 调用）。关键节点：1) `sys_openat` 解析路径并分配 fd；2) `lookup_path` 处理绝对/相对路径；3) `dirlookup` 逐级查找目录项；4) FAT32 `fat_lookup_dir` 读取磁盘目录。	1. sys_openat() (kernel/syscall/sysfile.c) - syscall 入口 2. namei()/nameifrom() (kernel/fs/fs.c) - VFS 路径解析 3. dirlookup() (kernel/fs/fs.c) - VFS 目录查找 4. fat_lookup_dir() (kernel/fs/fat32/dirent.c) - FAT32 具体实现 5. fat_alloc_entry() (kernel/fs/fat32/dirent.c) - FAT32 文件分配
证据	[1] `kernel/syscall/sysfile.c` · sys_openat `uint64 sys_openat(void) { ... ip = nameifrom(dp, path); ... }` [2] `kernel/fs/fs.c` · lookup_path `static struct inode *lookup_path(struct inode ip, char *path, int parent, char name)` [3] `kernel/fs/fs.c` · dirlookup `struct inode *dirlookup(struct inode dir, char *filename, uint poff)`	[1] `kernel/syscall/sysfile.c` · sys_openat `uint64 sys_openat(void) { ... if((ip = nameifrom(dp, path)) == NULL) ... if((f = filealloc()) == NULL || (fd = fdalloc(f, omode & O_CLOEXEC)) < 0)...` [2] `kernel/fs/fs.c` · nameifrom `struct inode *nameifrom(struct inode ip, char *path) { char name[MAXNAME + 1]; return lookup_path(ip, path, 0, name); }` [3] `kernel/fs/fs.c` · dirlookup `struct inode * dirlookup(struct inode *dir, char filename, uint *poff) { ... ip = dir->op->lookup(dir, filename, poff); ...` [4] `kernel/fs/fat32/dirent.c` · fat_lookup_dir `struct inode * fat_lookup_dir(struct inode *dir, char filename, uint *poff) { ... struct fat32_entry ep = fat_lookup_dir_ent(dir, filename, &off); ...`

`Q05_005` · `short_answer`

> 文件描述符表 (File Descriptor Table, FD Table) 的实现形态是什么？（固定数组/Vec/BTreeMap 等；必须给结构体定义证据）

项目	方法A	方法B
答案	固定数组 + 链表扩展形态。定义于 `include/fs/file.h:32-38`：`struct fdtable { uint16 basefd; uint16 nextfd; uint16 used; uint16 exec_close; struct file *arr[NOFILE]; struct fdtable *next; }`。主表为固定大小数组 `arr[NOFILE]`，通过 `next` 指针支持链表扩展。	链表 + 数组的混合结构。struct fdtable 包含固定大小数组 arr[NOFILE]，并通过 next 指针链接多个 fdtable 形成链表以支持扩展。

项目	方法A	方法B
证据	[1] `include/fs/file.h` · fdtable `struct fdtable { uint16 basefd; ... struct file arr[NOFILE]; struct fdtable next; }`	[1] `include/fs/file.h` · fdtable `struct fdtable { uint16 basefd; uint16 nextfd; uint16 used; uint16 exec_close; struct file arr[NOFILE]; struct fdtable next; };`

`Q05_007` · `short_answer`

> 若存在缓存：驱逐策略是什么（LRU/Clock/FIFO/无驱逐）？必须指出判断依据（字段/算法分支）证据。

项目	方法A	方法B
答案	LRU（最近最少使用）驱逐策略。证据：`kernel/fs/bio.c:88-118` 中 `bget()` 使用 `lru_head` 双向链表管理空闲 buffer，新分配的 buffer 从链表尾部（最久未使用）获取（`struct d_list *dl = lru_head.prev`）；`bput()` 将释放的 buffer 加回链表头部（`dlist_add_after(&lru_head, &b->list)`），形成 LRU 队列。	LRU (Least Recently Used)。使用 lru_head 双向链表，新访问的 buf 移到链表头部，驱逐时从尾部取最久未使用的 buf。通过 lrucnt[] 数组记录访问计数辅助判断。
证据	[1] `kernel/fs/bio.c` · bget `struct d_list *dl = lru_head.prev; ... b = container_of(dl, struct buf, list);` [2] `kernel/fs/bio.c` · bput `dlist_add_after(&lru_head, &b->list);`	[1] `kernel/fs/bio.c` · bget `struct d_list dl = lru_head.prev; if (dl != &lru_head) { b = container_of(dl, struct buf, list); ... dlist_del(&b->list); }` *[2] `kernel/fs/fat32/fat.c` · fatcache* `struct { char page; int allocidx; uint32 fatsec[FAT_CACHE_NSEC]; uint32 lrucnt[FAT_CACHE_NSEC]; int8 dirty[FAT_CACHE_NSEC]; } fatcache;`

`Q05_017` · `single_choice`

> 文件数据块分配方式 (File Allocation Method, Stallings Ch12) 更接近哪种?

项目	方法A	方法B
答案	FAT 表内嵌空闲链（FAT32 特有）	B. 链式分配 (Chained/Linked Allocation)：块通过指针链接
证据	[1] `kernel/fs/fat32/fat32.c` · fat32_sb `FAT32 使用 FAT 表记录簇链，通过 fat->bpb.fat_sz 和 fat->free_count 管理空闲簇`	[1] `kernel/fs/fat32/fat.c` · read_fat `uint32 read_fat(struct superblock sb, uint32 cluster) { ... uint32 fat_sec = fat_sec_of_clus(fat, cluster, 1); uint32 sec_off = fat_offset_of_clus(fat, cluster); ... next = (uint32 )(fat->fatcache.page + idx * SECSZ + sec_off); ...` *[2] `kernel/fs/fat32/cluster.c` · reloc_clus* `uint32 reloc_clus(struct inode ip, uint off, int alloc) { ... clus = read_fat(sb, preclus); // get next cluster number ... table->clus[i] = clus; ...`

`Q05_021` · `single_choice`

> 设备发现/枚举机制更接近哪种?

项目	方法A	方法B
答案	混合（多种并存）	C. 硬编码设备表/固定 MMIO 地址

项目	方法A	方法B
证据	[1] include/memlayout.h · UART #define UART 0x10000000L (QEMU) / 0x38000000L (k210) 硬编码地址 [2] bootloader/SBI/rustsbi-qemu/src/main.rs · count_harts 使用 device_tree 解析 DTB 获取 CPU 核心数	[1] include/memlayout.h · MMIO_addresses #define UART 0x38000000L #define CLINT 0x02000000L #define PLIC 0x0c000000L #ifndef QEMU #define GPIOHS 0x38001000 #define DMAC 0x50000000 #define GPIO 0x50200000 #define SPI0 0x52000000 #define SYSCTL 0x50440000 #endif [2] kernel/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { ... #ifndef QEMU fpioa_pin_init(); dmac_init(); #endif disk_init(); binit(); ...

Q05_022 · tri_state_impl

> 是否能在代码中证实解析了 .dtb /DeviceTree？（必须三态；若 implemented 必须指出解析入口）

项目	方法A	方法B
答案	implemented	stub
证据	[1] bootloader/SBI/rustsbi-qemu/src/main.rs · count_harts unsafe fn count_harts(dtb_pa: usize) { use device_tree::{DeviceTree, Node}; ... if let Ok(dt) = DeviceTree::load(data) { ... }	[1] bootloader/SBI/rustsbi-qemu/src/main.rs · count_harts unsafe fn count_harts(dtb_pa: usize) -> usize { use device_tree::{DeviceTree, Node}; ... let header = &*(dtb_pa as const DtbHeader); let magic = u32::from_be(header.magic); if magic == DEVICE_TREE_MAGIC { let size = u32::from_be(header.size); let data = core::slice::from_raw_parts(dtb_pa as *const u8, size as usize); if let Ok(dt) = DeviceTree::load(data) { if let Some(cpu_map) = dt.find("/cpus/cpu-map") { return enumerate_cpu_map(cpu_map) } } } [2] kernel/main.c · main void main(unsigned long hartid, unsigned long dtb_pa) { ... // dtb_pa is passed but not used for device discovery on K210 #ifndef QEMU fpioa_pin_init(); dmac_init(); #endif

Q05_023 · short_answer

> 驱动框架接口是什么？（Rust Driver trait / C driver ops / 注册表；必须引用接口定义证据）

项目	方法A	方法B
答案	无统一驱动框架接口。驱动直接在 kernel/main.c:main() 中顺序初始化（disk_init() → binit() → plicinit() 等），无 driver trait/ops 注册表机制。块设备通过 disk_read() / disk_write() 函数指针间接调用具体驱动（sdcard/virtio）。	C 函数接口，无统一驱动框架。各驱动通过初始化函数直接调用（如 sdcard_init()、disk_init()），无 driver trait 或注册表机制。
证据	[1] kernel/main.c · main disk_init(); binit(); plicinit(); ... [2] kernel/hal/disk.h · disk_ops 无统一 ops 表，直接调用 disk_read/disk_write	[1] include/hal/sdcard.h · sdcard_init void sdcard_init(void); void sdcard_read_sector(uint8 *buf*, *int sectorno); void sdcard_write_sector(uint8* buf, int sectorno); [2] include/hal/disk.h · disk_init void disk_init(void); int disk_read(struct buf *b); void disk_write(struct buf* b); int disk_submit(struct buf *b);

Q05_024 · short_answer

> 驱动注册与初始化顺序是什么？（init_drivers/probe/driver_manager 等；列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	初始化顺序（`kernel/main.c:43-62`）：1) `consoleinit()` (UART 早期输出)；2) `kpminit()` (物理内存管理)；3) `kvminit()` (内核页表)；4) `plicinit()` (中断控制器)；5) `disk_init()` (块设备驱动初始化)；6) `binit()` (块缓存)。无 driver_manager/probe 机制。	1. cpuinit() - CPU 初始化 2. consoleinit() - 控制台初始化 3. kpminit() - 物理内存初始化 4. kvminit()/kvminithart() - 页表初始化 5. disk_init() - 磁盘驱动初始化 6. binit() - 缓冲缓存初始化 所有驱动在 kernel/main.c 的 main() 函数中顺序调用，无动态注册。
证据	[1] `kernel/main.c` · main `consoleinit(); ... plicinit(); disk_init(); binit();`	[1] `kernel/main.c` · main `void main(unsigned long hartid, unsigned long dtb_pa) { ... cpuinit(); floatinithart(); consoleinit(); printfinit(); kpminit(); kvminit(); kvminithart(); ... #ifndef QEMU fpioa_pin_init(); dmac_init(); #endif disk_init(); binit(); ... userinit(); ...`

`Q05_029` · `short_answer`

> MMIO 地址来源是什么？（DTB 提供 / 常量硬编码 / 物理→虚拟转换；必须给证据）

项目	方法A	方法B
答案	常量硬编码。定义于 `include/memlayout.h:36-82`，如 `#define UART 0x10000000L` (QEMU) / `0x38000000L` (k210)，`#define VIRTIO0 0x10001000`，`#define PLIC 0x0c000000L`。通过 `VIRT_OFFSET` 转换为虚拟地址。	主要是常量硬编码，部分通过 VIRT_OFFSET 进行物理→虚拟转换。K210 平台使用硬编码地址（如 UART 0x38000000L），QEMU 平台部分地址通过设备树获取。
证据	[1] `include/memlayout.h` · UART `#define UART 0x10000000L (QEMU) / 0x38000000L (k210)` [2] `include/memlayout.h` · VIRT_OFFSET `#define VIRT_OFFSET 0x3F00000000L`	[1] `include/memlayout.h` · MMIO_definitions `#ifdef QEMU #define UART 0x10000000L #else #define UART 0x38000000L #endif #define UART_V (UART + VIRT_OFFSET) #define CLINT 0x02000000L #define CLINT_V (CLINT + VIRT_OFFSET) #define PLIC 0x0c000000L #define PLIC_V (PLIC + VIRT_OFFSET)` [2] `include/memlayout.h` · VIRT_OFFSET `#define VIRT_OFFSET 0x3F00000000L`

`Q05_030` · `short_answer`

> 多平台适配是如何通过构建/条件编译选择驱动的？（features/Kconfig/Makefile 规则；必须给证据）

项目	方法A	方法B
答案	Makefile 条件编译。`Makefile:1-28` 定义 `platform := k210` 或 `platform := qemu`，通过 `CFLAGS += -D QEMU` 切换平台。`include/memlayout.h:36-40` 使用 `#ifdef QEMU` 区分 UART/VIRTIO 地址。	通过 Makefile 的 platform 变量和 C 语言的 #ifdef QEMU 条件编译。Makefile 中设置 platform := k210 或 platform := qemu，源码中通过 #ifdef QEMU 区分不同平台的驱动和地址定义。

项目	方法A	方法B
证据	[1] `Makefile` · platform `platform := k210 / qemu, CFLAGS += -D QEMU` [2] `include/memlayout.h` · QEMU `#ifdef QEMU ... #else ... #endif`	[1] `Makefile` · platform `platform := k210 # platform := qemu ... ifeq ($(platform), qemu) CFLAGS += -D QEMU endif ... ifeq ($(platform), k210) SBI := ./sbi/sbi-k210 else SBI := ./sbi/sbi-qemu endif` [2] `Makefile` · SRC `ifeq ($(platform), k210) SRC += $K/hal/spi.c $K/hal/gpiohs.c $K/hal/fpioa.c $K/hal/sdcard.c $K/hal/dmac.c $K/hal/sysctl.c $K/utils/utils.c else SRC += $K/hal/virtio_disk.c endif` [3] `include/memlayout.h` · QEMU_check `#ifdef QEMU #define UART 0x10000000L #else #define UART 0x38000000L #endif ... #ifndef QEMU #define GPIOHS 0x38001000 #define DMAC 0x50000000 ... #endif`

`Q05_031` · `tri_state_impl`

> 是否存在 MMU 启用前后串口地址切换（phys/virt 切换）逻辑？（必须三态）

项目	方法A	方法B
答案	implemented	not_found
证据	[1] `include/memlayout.h` · UART_V `#define UART_V (UART + VIRT_OFFSET)` [2] `kernel/mm/vm.c` · kvminit `kvmmap(UART_V, UART, PGSIZE, PTE_R | PTE_W);`	[1] `kernel/console.c` · uart_address_search 在附件源码中检索 `UART` 地址切换逻辑，`console.c` 中使用 `sbi_console_putchar()` 通过 `SBI` 调用输出，未直接使用 `MMIO` 地址。未找到 `MMU` 启用前后串口地址切换的显式逻辑。 [2] `include/memlayout.h` · uart_virt_search 虽然定义了 `UART_V = (UART + VIRT_OFFSET)`，但在实际代码中未找到 `MMU` 启用后使用 `UART_V` 而非 `UART` 的切换逻辑。

`Q05_033` · `single_choice`

> 块设备（磁盘/eMMC/NVMe）I/O 请求调度算法 (Scheduling Algorithm) (Disk Scheduling Algorithm) 更接近哪种？（Stallings Ch11；若无显式调度则选「FCFS 顺序提交」）

项目	方法A	方法B
答案	基于 virtio 环（queue 顺序提交，无显式磁盘调度）	A. FCFS（先来先服务 First-Come First-Served）
证据	[1] `kernel/hal/virtio_disk.c` · virtio_disk_rw 通过 `virtio descriptor ring` 顺序提交请求，无电梯算法 [2] `kernel/hal/sdcard.c` · sdcard_submit 请求按提交顺序进入 `wait_queue`，无重排序	[1] `kernel/hal/sdcard.c` · disk_scheduling_search 在附件源码中检索磁盘调度算法，`sdcard.c` 和 `virtio_disk.c` 中未找到 `SSTF`、`SCAN`、`C-SCAN` 等调度算法实现。请求按提交顺序处理，无显式调度逻辑。 [2] `kernel/fs/bio.c` · bwrite `void bwrite(struct buf *b) { ... res = disk_submit(b); b->valid = 1; releasesleep(&b->lock); if ((res & 0x1) == 0) bput(b); if ((res & 0x2) == 0) disk_write_start(); }`

06_sync_ipc — 同步互斥与进程间通信

`Q06_001` · `short_answer`

> 该内核提供了哪些同步原语？（SpinLock/Mutex/RwLock/Semaphore/Condvar/WaitQueue 等；列出类型定义证据）

项目	方法A	方法B
答案	SpinLock、SleepLock（阻塞型 Mutex）、WaitQueue。定义证据：SpinLock 定义于 `include/sync/spinlock.h:7-13`（struct spinlock 含 locked 字段）；SleepLock 定义于 `include/sync/sleeplock.h:9-16`（struct sleeplock 含 locked 字段与内部 spinlock）；WaitQueue 定义于 `include/sync/waitqueue.h:16-24`（struct wait_queue 含 spinlock 与双向链表头，struct wait_node 为等待节点）。实现文件：`kernel/sync/spinlock.c`（acquire/release）、`kernel/sync/sleeplock.c`（acquiresleep/releasesleep）。	SpinLock, SleepLock, WaitQueue 证据: 1. SpinLock: include/sync/spinlock.h 定义 struct spinlock { uint locked; char *name; struct cpu* cpu; } 2. SleepLock: include/sync/sleeplock.h 定义 struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; }* *3. WaitQueue: include/sync/waitqueue.h 定义 struct wait_queue { struct spinlock lock; struct d_list head; } 和 struct wait_node { void* chan; struct d_list list; }*
证据	[1] `include/sync/spinlock.h` · spinlock `struct spinlock { uint locked; char *name; struct cpu* cpu; };` [2] `include/sync/sleeplock.h` · sleeplock `struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; };` [3] `include/sync/waitqueue.h` · wait_queue `struct wait_queue { struct spinlock lock; struct d_list head; };`	[1] `include/sync/spinlock.h` · spinlock `struct spinlock { uint locked; char *name; struct cpu* cpu; };` [2] `include/sync/sleeplock.h` · sleeplock `struct sleeplock { uint locked; struct spinlock lk; char *name; int pid; };` *[3]* `include/sync/waitqueue.h` · *wait_queue* `struct wait_queue { struct spinlock lock; struct d_list head; }; struct wait_node { void* chan; struct d_list list; };`

`Q06_004` · `fill_in`

> sleep / wakeup 不变量 (Sleep-Wakeup Invariant) 分析，按格式填写： - sleep 入口函数: ***（路径） - 入睡前持有的锁:***（无则写 none） - 防丢 wakeup (Lost Wakeup Prevention) 机制: ***（如：持队列锁检查条件 / 无防护） - wakeup 函数:***（路径） - 唤醒与锁释放顺序: ___（先唤醒后释放 / 先释放后唤醒 / 其他）

项目	方法A	方法B
答案	sleep 入口函数: `kernel/sched/proc.c:582` (sleep(void *chan, struct spinlock* lk)) 入睡前持有的锁: proc_lock（通过__enter_proc_cs 获取）+ 调用者传入的 lk（先释放后在 sleep 返回后重新获取） 防丢 wakeup (Lost Wakeup Prevention) 机制: 持 proc_lock 检查条件并调用__insert_sleep() 将进程加入睡眠队列，确保在释放 lk 前已完成入队，wakeup 持 proc_lock 遍历睡眠队列，避免丢失唤醒 wakeup 函数: `kernel/sched/proc.c:392` (wakeup(void *chan)) 唤醒与锁释放顺序: 先唤醒（__wakeup_no_lock 在 proc_lock 保护下执行）后释放（__leave_proc_cs 释放 proc_lock），符合 Stallings 描述的防丢 wakeup 不变量	sleep 入口函数：kernel/sched/proc.c::sleep() 入睡前持有的锁：调用者传入的 lk（自旋锁） 防丢 wakeup (Lost Wakeup Prevention) 机制：持队列锁检查条件 / 无防护（在 sleep() 中先 acquire 进程锁，然后插入睡眠队列，再调用 sched() 释放 CPU） wakeup 函数：kernel/sched/proc.c::wakeup() 唤醒与锁释放顺序：先唤醒后释放（wakeup 遍历 proc_sleep 队列，将匹配 chan 的进程移到 runnable 队列，然后被唤醒的进程在 scheduler 中获得 CPU 后继续执行）
证据	[1] `kernel/sched/proc.c` · sleep `行 582-606：if (&proc_lock != lk) { acquire(&proc_lock); release(lk); } ... __insert_sleep(p); sched(); ... release(&proc_lock); acquire(lk);` [2] `kernel/sched/proc.c` · wakeup `行 392-405：acquire(&proc_lock); int flag = __wakeup_no_lock(chan); release(&proc_lock); if (flag && avail) { sbi_send_ipi(...); }` [3] `kernel/sched/proc.c` · lock ordering comment `行 249-253：NOTICE! To avoid any potential deadlock with proc_lock, proc_lock should be acquired last`	[1] `kernel/sched/proc.c` · sleep `void sleep(void *chan, struct spinlock* lk) { struct proc *p = myproc(); acquire(&p->lk); p->chan = chan; __remove(p); __insert_sleep(p); release(&p->lk); sched(); acquire(&p->lk); p->chan = NULL; release(&p->lk); }` *[2] `kernel/sched/proc.c` · wakeup* `void wakeup(void *chan) { struct proc *p = proc_sleep; while (NULL != p) { struct proc* next = p->sched_next; if ((uint64)chan == (uint64)p->chan) { __remove(p); p->timer = TIMER_IRQ; p->chan = NULL; __insert_runnable(PRIORITY_IRQ, p); } p = next; } }`

`Q06_011` · `short_answer`

> 若实现 signal handler：用户态 handler 上下文如何构建？是否存在 sigreturn 恢复原 trap frame？（必须给证据）

项目	方法A	方法B
答案	上下文构建：在 `kernel/sched/signal.c:sighandle()`（行 178-258）中，内核分配 `struct sig_frame`（含 trapframe 指针、信号掩码、signum），保存当前用户态 trapframe 到新分配的内存，修改 p->trapframe 指向新的陷阱帧，设置 epc 为 sig_trampoline 地址，然后返回用户态执行 trampoline。sigreturn 存在：`kernel/sched/signal.c:263-283` 实现 `sigreturn()`，从 p->sig_frame 链表取出保存的原 trapframe，恢复 p->trapframe，释放 sig_frame 结构，完成上下文恢复。	信号处理上下文构建： 1. 在 kernel/sched/signal.c 的 sighandle() 中，当检测到信号时，分配 sig_frame 结构保存当前 trapframe 2. 修改 trapframe->epc 指向 sig_trampoline 中的 sig_handler 3. 设置 trapframe->a0 为信号编号，trapframe->a1 为 handler 地址 sigreturn 恢复： 1. kernel/trap/sig_trampoline.S 中 sig_handler 执行完用户 handler 后执行 SYS_rt_sigreturn 2. kernel/sched/signal.c 的 sigreturn() 从 sig_frame 链表中恢复之前的 trapframe 3. 释放 sig_frame 并返回到原执行点

项目	方法A	方法B
证据	[1] `kernel/sched/signal.c` · sighandle `行 178-258：分配 sig_frame，保存原 trapframe，设置 trampoline 入口` [2] `kernel/sched/signal.c` · sigreturn `行 263-283：p->trapframe = frame->tf; p->sig_frame = frame->next; kfree(frame);` [3] `kernel/trap/sig_trampoline.S` · sig_handler `jalr a1 调用用户 handler，然后 li a7, SYS_rt_sigreturn; ecall`	[1] `kernel/sched/signal.c` · sighandle `void sighandle(void) { struct sig_frame *frame = kmalloc(sizeof(struct sig_frame)); struct trapframe tf = kmalloc(sizeof(struct trapframe)); frame->tf = p->trapframe; tf->epc = (uint64)(SIG_TRAMPOLINE + ((uint64)sig_handler - (uint64)sig_trampoline)); tf->a0 = signum; tf->a1 = (uint64)(sigact->sigact.__sigaction_handler.sa_handler); p->trapframe = tf; frame->next = p->sig_frame; p->sig_frame = frame; }` [2] `kernel/sched/signal.c` · sigreturn `void sigreturn(void) { struct proc p = myproc(); struct sig_frame frame = p->sig_frame; kfree(p->trapframe); p->trapframe = frame->tf; p->sig_frame = frame->next; kfree(frame); }` [3] `kernel/trap/sig_trampoline.S` · sig_handler `sig_handler: jalr a1; li a7, SYS_rt_sigreturn; ecall`

`Q06_013` · `single_choice`

> 底层原子操作来源更接近哪种？

项目	方法A	方法B
答案	自定义汇编（ldxr/stxr、lock xchg 等）	A. Rust core::sync::atomic（标准库）
证据	[1] `kernel/sync/spinlock.c` · acquire `行 34：while(__sync_lock_test_and_set(&lk->locked, 1) != 0); 使用 GCC 内置原子操作，RISC-V 后端生成 amoswap.w.aq 指令` [2] `kernel/sync/spinlock.c` · release `行 71：__sync_lock_release(&lk->locked); 生成 amoswap.w 指令` [3] `kernel/sync/spinlock.c` · atomic comment `行 31-33 注释说明：On RISC-V, sync_lock_test_and_set turns into an atomic swap`	[1] `kernel/sync/spinlock.c` · acquire `while(__sync_lock_test_and_set(&lk->locked, 1) != 0); __sync_synchronize();` [2] `kernel/sync/spinlock.c` · release `__sync_synchronize(); __sync_lock_release(&lk->locked);`

`Q06_014` · `short_answer`

> 死锁四必要条件（Coffman Conditions）在该内核中是否均成立？ 请逐条作答（互斥 Mutual Exclusion / 持有并等待 Hold-and-Wait / 不可剥夺 No Preemption / 循环等待 Circular Wait），并结合 SpinLock/Mutex 的实现给出证据或写「不适用」。

项目	方法A	方法B
答案	1. 互斥 (Mutual Exclusion): 成立。SpinLock 通过原子交换指令保证同一时刻仅一个 CPU 持有锁（`kernel/sync/spinlock.c:34` amoswap.w.aq）；SleepLock 在 SpinLock 基础上增加睡眠语义，同样保证互斥。 2. 持有并等待 (Hold-and-Wait): 成立。`kernel/sched/proc.c:582-606` 的 sleep() 允许进程持有 lk 锁的同时释放 proc_lock 并进入睡眠，唤醒后重新获取 lk，存在持有资源等待其他资源的场景。 3. 不可剥夺 (No Preemption): 成立。SpinLock 持有期间不能被强制剥夺（只能由持有者主动 release）；SleepLock 持有者睡眠时锁仍被占用，其他进程只能等待。 4. 循环等待 (Circular Wait): 可能成立。内核存在多锁嵌套场景（如 pipe 操作同时持有 pi->lock 和 wait_queue->lock），但通过锁顺序规范预防（见 Q06_016）。	1. 互斥 (Mutual Exclusion): 成立 证据：spinlock.c 中 acquire() 使用 __sync_lock_test_and_set 确保同一时刻只有一个 CPU 能获得锁 2. 持有并等待 (Hold-and-Wait): 成立 证据：proc.c 中 sleep() 函数显示进程可以在持有锁的情况下进入睡眠等待其他资源 3. 不可剥夺 (No Preemption): 成立 证据：spinlock.c 中 release() 必须由持有锁的 CPU 主动调用，内核不会强制剥夺锁 4. 循环等待 (Circular Wait): 可能成立 证据：代码中未发现全局锁顺序规范，多个锁的获取顺序未强制规定，存在循环等待可能性

项目	方法A	方法B
证据	[1] `kernel/sync/spinlock.c` · acquire 行 23-42：原子交换指令实现互斥 [2] `kernel/sched/proc.c` · sleep 行 582-606：持锁进入睡眠，唤醒后重新获取 [3] `kernel/sched/proc.c` · deadlock prevention 行 249-253/454-458/603-605：多处注释说明锁顺序以避免死锁	[1] `kernel/sync/spinlock.c` · acquire `void acquire(struct spinlock lk) { push_off(); while(__sync_lock_test_and_set(&lk->locked, 1) != 0); __sync_synchronize(); lk->cpu = mycpu(); }` *[2] kernel/sched/proc.c · sleep* `void sleep(void chan, struct spinlock lk) { acquire(&p->lk); p->chan = chan; __remove(p); __insert_sleep(p); release(&p->lk); sched(); }` *[3] kernel/sync/spinlock.c · release* `void release(struct spinlock lk) { __sync_synchronize(); __sync_lock_release(&lk->locked); pop_off(); }`

`Q06_015` · `single_choice`

> 内核对死锁 (Deadlock) 的处理策略更接近哪种?

项目	方法A	方法B
答案	死锁预防 (Deadlock Prevention)：通过锁顺序等消除 Coffman 必要条件	D. 忽略 (Ostrich Algorithm)：不处理，依赖外部重启
证据	[1] `kernel/sched/proc.c` · lock ordering 行 249-253：NOTICE! To avoid any potential deadlock with proc_lock, proc_lock should be acquired last [2] `kernel/sched/proc.c` · exit lock order 行 454-458：acquire proc_lock after parent's lock, to avoid deadlock with parent calling sleep(p, &p->lk) in wait4() [3] `kernel/sched/proc.c` · sleep lock order 行 603-605：release proc_lock first to avoid deadlock in case another call to sleep() with the same lk	[1] `kernel` · deadlock_detection 在整个 kernel 目录中搜索死锁检测、避免或恢复相关代码，未找到 deadlock detection/avoidance/recovery 实现 [2] `kernel/sched/proc.c` · lock_ordering 在 proc.c 中未发现锁顺序检查或死锁预防机制

`Q06_016` · `tri_state_impl`

> 是否存在全局锁顺序（Lock Ordering）规范或注释，以预防嵌套锁导致的循环等待死锁 (Circular Wait Deadlock)？ （必须三态；若 implemented 需给出锁排序规则或 ABBA 锁检测代码证据）

项目	方法A	方法B
答案	implemented	not_found
证据	[1] `kernel/sched/proc.c` · proc_lock ordering 行 249-253：proc_lock should be acquired last with any situation requiring multiple spinlocks [2] `kernel/sched/proc.c` · exit lock order 行 454-458：acquire proc_lock after parent's lock, to avoid deadlock with parent calling sleep(p, &p->lk) in wait4() [3] `kernel/sched/proc.c` · sleep lock release order 行 603-605：release proc_lock first to avoid deadlock in case another call to sleep() with the same lk	[1] `kernel` · lock_order 在整个 kernel 目录中搜索 lock ordering/lock order/锁顺序 相关注释或规范，未找到全局锁顺序定义 [2] `include/sync` · lock_hierarchy 在 sync 头文件中搜索锁层次结构或锁获取顺序规范，未找到相关文档

`Q06_017` · `tri_state_impl`

> 是否实现管程/条件变量 (Monitor / Condition Variable, Stallings Ch5)？ （必须三态；搜索 Condvar / condition_variable / monitor / wait/notify/signal 等；若 implemented 需区分 Hoare 语义（等待者立即恢复）vs Mesa 语义（等待者重新竞争

> 锁））

项目	方法A	方法B
答案	not_found	implemented
证据	[1] `repos/xv6-k210` · condvar/condition_variable `grep 搜索 'condvar|condition_variable|Condition|notify|wait.*notify'` 仅找到无关匹配（`license` 头文件、`VIRTIO_MMIO_QUEUE_NOTIFY` 等），无条件变量实现 [2] `include/sync/` · sync headers 仅含 `spinlock.h`、`sleeplock.h`、`waitqueue.h`，无 `condvar.h` 或类似定义	[1] `kernel/sched/proc.c` · sleep `void sleep(void *chan, struct spinlock* lk) { struct proc *p = myproc(); acquire(&p->lk); p->chan = chan; __remove(p); __insert_sleep(p); release(&p->lk); sched(); acquire(&p->lk); p->chan = NULL; release(&p->lk); }` *[2] `kernel/sched/proc.c` · wakeup* `void wakeup(void *chan) { struct proc *p = proc_sleep; while (NULL != p) { struct proc* next = p->sched_next; if ((uint64)chan == (uint64)p->chan) { __remove(p); p->timer = TIMER_IRQ; p->chan = NULL; __insert_runnable(PRIORITY_IRQ, p); } p = next; } }` [3] `kernel/fs/pipe.c` · pipelock `static void pipelock(struct pipe *pi, struct wait_node* wait, int who) { struct wait_queue *q; q = (who == PIPE_READER) ? &pi->rqueue : &pi->wqueue; acquire(&q->lock); wait_queue_add(q, wait); while (!wait_queue_is_first(q, wait)) { sleep(wait->chan, &q->lock); } release(&q->lock); }`

`Q06_018` · `short_answer`

> 经典同步问题验证 (Classic Synchronization Problems, Stallings Ch5)： 以下三个经典问题在该内核中是否有对应实现或测试？ - 生产者-消费者 (Producer-Consumer / Bounded Buffer)： ***(implemented/not*found + 证据）** **- 读者-写者 (Readers-Writers)：**_（实现了读者优先/写者优先/公平？ + 证据） - 哲学家就餐 (Dining Philosophers)：___（implemented/not_found）

项目	方法A	方法B
答案	生产者 - 消费者 (Producer-Consumer / Bounded Buffer)：not_found（grep 搜索 'producer.*consumer*|*bounded*.buffer' 未找到匹配；但 pipe 实现本质上是生产者 - 消费者模式，`kernel/fs/pipe.c` 使用环形缓冲 + 等待队列实现阻塞式读写，但未作为独立测试或示例代码存在） 读者 - 写者 (Readers-Writers)：not_found（grep 搜索 'reader.*writer*' *未找到匹配；无 RwLock 实现，仅通过 pipe 的读写分离等待队列间接支持，但非标准读者 - 写者锁）* *哲学家就餐 (Dining Philosophers)：not_found（grep 搜索 'dining*.philosoph' 未找到匹配）	生产者 - 消费者 (Producer-Consumer / Bounded Buffer)：implemented + 证据：kernel/fs/pipe.c 中的 pipe 实现使用环形缓冲区和 wait_queue 实现生产者 - 消费者模式，pipewrite 为生产者，piperead 为消费者 读者 - 写者 (Readers-Writers)：not_found + 证据：在代码中搜索 rwlock 或 readers_writers 相关实现，未找到专门的读者 - 写者锁实现 哲学家就餐 (Dining Philosophers)：not_found + 证据：在 xv6-user/ 目录下搜索 dining 或 philosophers 相关测试程序，未找到哲学家就餐问题测试

项目	方法A	方法B
证据	[1] `repos/xv6-k210` · classic problems `grep 搜索 'producer.consumer|bounded.buffer|reader.writer|dining.philosoph'` 未找到匹配 [2] `kernel/fs/pipe.c` · pipe as producer-consumer `pipe 读写分离 + 环形缓冲 + 阻塞语义，本质是生产者 - 消费者模式，但非独立示例`	[1] `kernel/fs/pipe.c` · pipewrite `int pipewrite(struct pipe pi, uint64 addr, int n) { while ((m = pi->nwrite - pi->nread) == PIPESIZE(pi)) { sleep(wait->chan, &pi->lock); } // write data pi->nwrite += count; pipewakeup(pi, PIPE_READER); }` [2] `kernel/fs/pipe.c` · piperead `int piperead(struct pipe pi, uint64 addr, int n) { while ((m = pi->nwrite - pi->nread) == 0) { sleep(wait->chan, &pi->lock); } // read data pi->nread += count; pipewakeup(pi, PIPE_WRITER); }` [3] `xv6-user` · classic_tests `在 xv6-user/ 目录下搜索 producer_consumer, readers_writers, dining_philosophers 相关测试文件，未找到专门实现`

07_security — 安全机制与权限模型

`Q07_002` · `tri_state_impl`

> 是否存在凭证/权限数据结构（UID/GID/Credential/Capability/ACL 等）？ （必须三态）

项目	方法A	方法B
答案	stub	implemented
证据	[1] `include/fs/stat.h` · kstat `struct kstat { ... uint32_t uid; uint32_t gid; ... };` [2] `include/sched/proc.h` · proc `struct proc { ... // 无 uid/gid/credential 字段 }` [3] `kernel/exec.c` · execve `{AT_UID, 0}, {AT_EUID, 0}, {AT_GID, 0}, {AT_EGID, 0} // 硬编码为 0`	[1] `include/fs/stat.h` · kstat `struct kstat { uint32 uid; uint32 gid; ... } - UID/GID fields exist in file stat structure` [2] `include/fs/fs.h` · inode `struct inode contains mode field for file permissions (S_IFMT, S_IFDIR, S_IFREG, etc.)`

`Q07_003` · `tri_state_impl`

> 是否能证实在 syscall 路径上真实执行了权限检查（open/exec/write 等）？ （必须三态；仅有字段不算 implemented）

项目	方法A	方法B
答案	not_found	stub

项目	方法A	方法B
证据	[1] `kernel/syscall/sysproc.c` · sys_getuid `uint64 sys_getuid(void) { return 0; }` [2] `kernel/exec.c` · execve `{AT_UID, 0}, {AT_EUID, 0}, {AT_GID, 0}, {AT_EGID, 0} // 无权限检查逻辑`	[1] `kernel/syscall/sysfile.c` · sys_faccessat `sys_faccessat checks file existence and mode bits but returns 0 for root user without actual permission enforcement - assumes user as root` [2] `kernel/syscall/sysfile.c` · sys_openat `openat checks file type (directory vs regular file) but no UID/GID based permission checking implemented` [3] `kernel/fs/fs.c` · create `File creation uses mode parameter but no actual owner/permission enforcement in FAT32 implementation`

`Q07_004` · `short_answer`

> 若存在权限检查：入口点与核心检查函数链路是什么？（列 2-5 个节点并给证据）

项目	方法A	方法B
答案	未发现权限检查链。grep 搜索 check_perm/inode_permission/access_check 无结果。sys_getuid 仅返回硬编码 0，execve 中 AT_UID/AT_GID 硬编码为 0，无真实权限检查函数调用。	Permission check chain (limited): sys_faccessat() -> nameifrom() -> dirlookup() -> fat_lookup_dir_ent() - but actual permission enforcement is stubbed (returns success for root)
证据	[1] `kernel/syscall/sysproc.c` · sys_getuid `uint64 sys_getuid(void) { return 0; }` [2] `kernel/exec.c:241-244` · auxvec `{AT_UID, 0}, {AT_EUID, 0}, {AT_GID, 0}, {AT_EGID, 0}`	[1] `kernel/syscall/sysfile.c` · sys_faccessat `int sys_faccessat(void) { ... ip = nameifrom(ip, path); ... ilock(ip); ip->op->getattr(ip, &st); ... if ((imode & mode) != mode) return -1; } - checks mode bits but assumes root` [2] `kernel/fs/fs.c` · nameifrom `struct inode *nameifrom(struct inode ip, char *path) - traverses directory tree to find inode` [3] `kernel/fs/fat32/dirent.c` · fat_lookup_dir_ent `Looks up directory entry in FAT32 filesystem, returns inode with mode information`

`Q07_007` · `tri_state_impl`

> 是否存在栈保护/溢出防护（stack canary/guard page）或等价机制？（必须三态）

项目	方法A	方法B
答案	stub	implemented
证据	[1] `include/memlayout.h:113` · guard_page_comment `// each surrounded by invalid guard pages.` [2] `kernel/mm/vm.c:596` · stack_guard_comment `// used by exec for the user stack guard page.`	[1] `Makefile` · stack_protector `CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector) - stack protector explicitly disabled but...` [2] `include/memlayout.h` · KSTACK `// map kernel stacks beneath the trampoline, each surrounded by invalid guard pages.` 保护页的 `PTE` 无效（即，`PTE_V` 置 `0`），因此，如果内核溢出内核堆栈，则很可能会导致异常 [3] `doc/内核原理 - 内存管理.md` · stack_protection 内核堆栈页面。每个进程都有自己的内核堆栈，该堆栈被映射到较高的位置，因此 `xv6` 在其下方可以留下未映射的保护页。保护页的 `PTE` 无效，如果内核溢出内核堆栈，则很可能会导致异常

`Q07_009` · `short_answer`

> 本项目支持哪些架构（riscv64/aarch64/x86_64/loongarch64 等）？每种架构的安全相关初始化（特权级配置、

> PMP/MPU/SMEP 等）是否有代码证据？（必须逐架构作答，无证据写「未发现」）

项目	方法A	方法B
答案	仅支持 riscv64 架构。证据：Makefile 中 TOOLPREFIX=riscv64-unknown-elf；bootloader/SBI/rustsbi-k210/.cargo/config.toml 中 target="riscv64gc-unknown-none-elf"。特权级隔离通过 RISC-V SSTATUS_SPP 位实现（include/hal/riscv.h），用户/内核态切换通过 sret 指令（kernel/trap/trampoline.S）。未发现 PMP/MPU 配置代码。	仅支持 riscv64 架构。安全初始化证据：1) RISC-V S-mode/U-mode 特权级分离（include/hal/riscv.h 的 SSTATUS_SPP）；2) 页表权限位 PTE_U 控制用户访问（include/hal/riscv.h 的 PTE_U）；3) K210 使用 RISC-V 1.9.1 规范，通过 RustSBI 处理 sfence.vma 异常来设置分页（doc/构建调试 - 外部中断 v2.md）；4) QEMU 和 K210 平台都有 trap 初始化（kernel/trap/trap.c 的 trapinithart）
证据	[1] `Makefile:11` · TOOLPREFIX `TOOLPREFIX := riscv64-unknown-elf-` [2] `bootloader/SBI/rustsbi-k210/.cargo/config.toml` · target `target = "riscv64gc-unknown-none-elf"` [3] `include/hal/riscv.h` · SSTATUS_SPP `#define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User`	[1] `Makefile` · toolchain `TOOLPREFIX := riscv64-unknown-elf- | CFLAGS += -march=rv64imafdc - RISC-V 64-bit architecture only` [2] `include/hal/riscv.h` · privilege_bits `#define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User | #define PTE_U (1L << 4) // 1 -> user can access` [3] `doc/构建调试 - 外部中断 v2.md` · k210_riscv_version `K210 采用 1.9 版本的 RISC-V 标准，不存在 sfence.vma 这条指令，只有旧版指令 sfence.vm...RustSBI 捕获 sfence.vma 异常，设置 mstatus.vm 位开启分页` [4] `kernel/trap/trap.c` · trapinithart `void trapinithart(void) { w_stvec((uint64)kernelvec); w_sstatus(r_sstatus() | SSTATUS_SIE); w_sie(r_sie() | SIE_SEIE | SIE_SSIE | SIE_STIE); } - sets up trap vector and enables interrupts`

`Q07_011` · `tri_state_impl`

> 是否实现了内核/用户页表隔离 (Kernel/User Page Table Isolation, KPTI 或等价机制)？ （x86: CR3 KPTI / SMEP / SMAP；RISC-V: PMP / S-mode 分离；AArch64: TTBR0/TTBR1 隔离； 必须三态；无则写未发现并列出已搜关键字）

项目	方法A	方法B
答案	implemented	not_found
证据	[1] `include/memlayout.h` · TRAMPOLINE `#define TRAMPOLINE (MAXVA - PGSIZE) // map the trampoline page to the highest address` [2] `kernel/trap/trampoline.S` · uservec `# this code is mapped at the same virtual address (TRAMPOLINE) in user and kernel space` [3] `kernel/trap/trap.c` · usertrapret `p->trapframe->kernel_satp = r_satp(); // kernel page table ... w_satp(MAKE_SATP(p->pagetable)); // user page table` [4] `include/hal/riscv.h` · SSTATUS_PUM `#define SSTATUS_PUM (1L << 18) // 控制用户态访问内核内存`	[1] `doc/内核设计 - 页表映射.md` · page_table_design `在 xv6-k210 这个较为简单的内核上，为了提高效率，我们取消了原本的用户、内核页表分离的规则，将用户空间与内核空间合并在一个页表中 - KPTI explicitly NOT implemented` [2] `kernel/mm/vm.c` · kvmcreate `pagetable_t kvmcreate() creates kernel page table that is copied for each process - user and kernel share same page table structure` [3] `include/memlayout.h` · address_layout `User space: 0x1000 to 0x80000000, Kernel space: 0x80000000 and above - both mapped in same page table with PTE_U flag controlling access` [4] `include/hal/riscv.h` · kpti_keywords `Searched for KPTI, CR3 switch, SMEP, SMAP, TTBR0/TTBR1 - none found. Only PTE_U flag for user/kernel separation`

`Q07_012` · `short_answer`

> UID/GID 字段是否在 syscall 路径上真实执行权限检查？（搜索 check_perm/inode_permission；若只有字段无检查链须标注「仅有定义但未强制执行 🔸」；给检查链证据或写「字段存在但无检查链」）

项目	方法A	方法B
答案	字段存在但无检查链。include/fs/stat.h 中 kstat 结构体有 uid/gid 字段，但 include/sched/proc.h 中 proc 结构体无 UID/GID 凭证字段。kernel/exec.c 中 AT_UID/AT_GID 硬编码为 0。grep 搜索 check_perm/inode_permission 无结果。sys_getuid 仅返回 0（🔸 桩函数）。	字段存在但无检查链 - UID/GID 字段定义在 struct kstat 中（include/fs/stat.h），但在 syscall 路径上没有基于 UID/GID 的实际权限强制执行。sys_faccessat 仅检查 mode 位且假设用户为 root（kernel/syscall/sysfile.c 注释：// assume user as root）
证据	[1] `include/fs/stat.h` · kstat `struct kstat { ... uint32_t uid; uint32_t gid; ... };` [2] `include/sched/proc.h` · proc `struct proc { ... // 无 uid/gid 字段 }` [3] `kernel/syscall/sysproc.c` · sys_getuid `uint64 sys_getuid(void) { return 0; }` [4] `kernel/exec.c:241-244` · auxvec `{AT_UID, 0}, {AT_EUID, 0}, {AT_GID, 0}, {AT_EGID, 0}`	[1] `include/fs/stat.h` · kstat `struct kstat { uint32 uid; uint32 gid; ... } - UID/GID fields defined` [2] `kernel/syscall/sysfile.c` · sys_faccessat `int sys_faccessat(void) { ... // assume user as root ... if ((imode & mode) != mode) return -1; } - comment explicitly states 'assume user as root', no actual UID/GID checking` [3] `kernel/sched/proc.h` · proc_credentials `struct proc does not contain uid/gid fields - no per-process credential tracking` [4] `kernel/fs/fs.c` · inode_permissions `inode structure has mode field but no owner uid/gid storage or enforcement in FAT32 implementation`

`Q07_013` · `single_choice`

> 访问控制模型 (Access Control Model, Stallings Ch15) 更接近哪种?

项目	方法A	方法B
答案	仅有特权级隔离（ring0/ring3），无细粒度访问控制	自主访问控制 DAC (Discretionary Access Control)：所有者自主设置权限（Unix 权限位）
证据	[1] `include/hal/riscv.h` · SSTATUS_SPP `#define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User` [2] `kernel/syscall/sysproc.c` · sys_getuid `return 0; // 无真实权限检查`	[1] `include/fs/stat.h` · permission_bits `#define S_IFMT 0170000 #define S_IFDIR 0040000 #define S_IFREG 0100000 - Unix-style file type and permission bits` [2] `kernel/syscall/sysfile.c` · sys_faccessat `Checks file mode bits (read/write/execute) against requested access - classic DAC model` [3] `kernel/fs/fat32/fat32.c` · fat_alloc_entry `File creation uses mode parameter (S_IFDIR, S_IFREG) but FAT32 doesn't store Unix permissions - permissions are virtual/in-memory only` [4] `doc/内核设计 - 文件系统 v2.md` · vfs_design `简单虚拟文件系统实现，基于 Linux VFS 概念但简化，支持 Unix-style operations like mount, but FAT32 underlying filesystem doesn't support full Unix permissions`

08_network — 网络子系统与协议栈

`Q08_004` · `short_answer`

> 选择一个发送路径（优先 sys_sendto），追踪：syscall → 协议栈 → 网卡驱动。列 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	未实现网络功能（❌ 未实现）。无法追踪发送路径，原因如下： 1. 无 sys_sendto 系统调用：include/sysnum.h 中无 SYS_sendto 定义 2. 无协议栈：全仓库 grep 未发现 tcp/udp/ip 等协议处理代码 3. 无网卡驱动：kernel/hal/ 仅有 virtio_disk.c（磁盘），无 virtio-net 或 e1000 等网卡驱动 4. 无 socket 抽象：include/fs/file.h 中文件类型仅支持普通文件/管道/设备，无 socket 类型	not_found: 本项目未实现网络子系统，不存在 sendto 系统调用、协议栈或网卡驱动路径。xv6-k210 专注于基础 OS 功能（文件系统、进程管理、内存管理），网络功能未实现。
证据	—	[1] `include/sysnum.h` · SYS_sendto `搜索 SYS_sendto 系统调用号，在 sysnum.h 中未找到定义。系统调用号最大到 SYS_msync，均为文件、进程、内存相关调用。` [2] `kernel/hal/disk.c` · disk driver `disk.c 实现了存储设备的读写接口（disk_read、disk_write），但这是针对 SD 卡/virtio 磁盘的驱动，不是网络网卡驱动。` [3] `kernel/hal/virtio_disk.c` · virtio driver `virtio_disk.c 实现了 virtio 块设备驱动（磁盘），有 virtio_disk_init、virtio_disk_rw 等函数，但没有 virtio_net 或任何网络相关的 virtio 驱动实现。`

09_debug_error — 调试机制与错误处理

`Q09_003` · `short_answer`

> panic 路径会输出哪些诊断？（寄存器 dump/栈回溯/停机；必须引用实现证据）

项目	方法A	方法B
答案	输出错误消息（含 CPU ID、文件路径、行号）、调用 backtrace() 打印栈帧返回地址、关闭中断并进入无限循环停机。无寄存器 dump。	Panic path outputs: panic message with module/file/line, stack backtrace via frame pointer walking, then halts in infinite loop. Register dump not implemented in kernel panic (only in trap handling prints scause/sepc/stval).
证据	[1] `kernel/printf.c` · **panic** `printf(__ERROR("panic"): "); printf(s); printf("\n"); backtrace(); intr_off(); for(;;);` **[2]** `include/printf.h` · **panic** `printf(__ERROR(__module_name )": hart %d at %s: %d\n", cpuid(), FILE, LINE); __panic(s);` [3] `kernel/printf.c` · backtrace `基于 FramePointer 遍历栈帧，打印每个帧的 ra 地址（返回地址减 4）`	[1] `kernel/printf.c` · **panic_backtrace** `__panic() calls printf() with error message including cpuid(), __FILE , LINE, then calls backtrace(). backtrace() walks frame pointers: while(fp < bottom) { printf("%p\n", ra-4); fp = *(fp-2); }. Then intr_off() and infinite loop.` [2] `kernel/trap/trap.c` · kerneltrap `On unhandled trap: printf("\nscause %p\n", scause); printf("sepc=%p stval=%p hart=%d\n", r_sepc(), r_stval(), r_tp()); then panic("kerneltrap").`

`Q09_005` · `tri_state_impl`

> 是否存在 **内核驻留的交互式监视器（kernel monitor）**？（对齐 Stallings《操作系统：精髓与设计原理》语境：**在内核态上下文**接受命令、用于探查/操控系统的监视器；**不包括**仅在用户态运行的常规 shell，如 `xv6-user/sh.c`、`user/` 下用户程序等——除非题面另有定义。必须三态；若 `implemented`：须给出 3–10 个 **用户可键入的 monitor 命令名** 及对应 **内核内** 解析/分发入口的 `路径:行号` 证据；仅以用户态 shell 充当内核 monitor 视为 **未切题** 应判 `stub` 或 `not_found` 并说明理由。）

项目	方法A	方法B
答案	implemented	not_found
证据	[1] `xv6-user/sh.c` · export `int export(char argv) - 支持 -p 打印所有环境变量或设置新环境变量` *[2] `xv6-user/sh.c` · runcmd* `void runcmd(struct cmd cmd) - 执行命令，支持 EXEC/REDIR/PIPE/LIST/BACK 等类型` [3] `xv6-user/sh.c` · parsecmd `struct cmd parsecmd(char) - 解析 shell 命令行` [4] `xv6-user/sh.c` · replace `int replace(char *buf) - 替换环境变量引用 $VAR`	[1] `xv6-user/sh.c` · shell `User-space shell implementation. Runs in user mode, parses commands like cd, export, exec. Not a kernel-resident monitor.` [2] `kernel/console.c` · consoleintr `Console input handler processes special keys (Ctrl-P prints process list, Ctrl-E prints file-entry list, Ctrl-K prints mem-alloc list, Ctrl-B prints buffer cache, Ctrl-Q prints physical memory). These are debug triggers, not interactive monitor commands with command parsing/dispatch.` [3] `kernel/trap/trap.c` · usertrap `No kernel monitor command parsing. Trap handling is for exceptions/interrupts/syscalls, not interactive command interpretation in kernel context.`

`Q09_007` · `short_answer`

> 错误码/错误类型体系是什么？（errno/Result/Error enum；给类型定义与传播点证据）

项目	方法A	方法B
答案	POSIX errno 风格宏定义（EPERM/ENOENT/ENOMEM 等），定义于 include/errno.h，无 Rust Result/Error enum。错误码通过系统调用返回值传播（负值表示错误）。	Uses POSIX-style errno constants defined in include/errno.h. System calls return negative errno values on error (e.g., -ENOENT, -EINVAL, -ENOMEM). Error propagation through syscall return values checked in user space.
证据	[1] `include/errno.h` · EPERM `#define EPERM 1 / Operation not permitted /` [2] `include/errno.h` · ENOENT `#define ENOENT 2 / No such file or directory /` [3] `include/errno.h` · ENOSYS `#define ENOSYS 38 / Invalid system call number /`	[1] `include/errno.h` · errno_definitions `Defines errno constants: EPERM=1, ENOENT=2, ESRCH=3, EINTR=4, EIO=5, ENOMEM=12, EACCES=13, EFAULT=14, EINVAL=22, etc. up to EADDRINUSE=98.` [2] `kernel/syscall/sysfile.c` · sys_openat `Returns -ENAMETOOLONG, -EBADF, -ENOENT, -EISDIR, -ENOMEM on various error conditions. Example: if((ip = nameifrom(dp, path)) == NULL) return -ENOENT;` [3] `kernel/syscall/sysmem.c` · sys_mmap `Returns -EINVAL, -EBADF, -EPERM on error. Example: if(off % PGSIZE || len == 0) return -EINVAL; if((int64)addr < 0) return -ENOMEM;`

`Q09_008` · `tri_state_impl`

> 是否存在 trace/perf/ftrace 等跟踪机制或 tracepoints？（必须三态）

项目	方法A	方法B
答案	implemented	stub

项目	方法A	方法B
证据	[1] `kernel/syscall/sysproc.c` · sys_trace `sys_trace(void) { myproc()->tmask = 1; return 0; }` [2] `xv6-user/strace.c` · main `strace 用户工具：调用 trace() 系统调用后 execve 执行目标程序`	[1] `include/sysnum.h` · SYS_trace `SYS_trace syscall number defined (value 18).` [2] `kernel/syscall/sysproc.c` · sys_trace `sys_trace() sets myproc()->tmask = 1. Enables syscall tracing for the process.` [3] `kernel/syscall/syscall.c` · syscall `If p->tmask is set, prints syscall name and arguments before execution, and return value after. Basic syscall tracing, not full ftrace/perf infrastructure with tracepoints.` [4] `xv6-user/strace.c` · strace `User-space strace utility that calls trace() syscall and execs target program. Traces syscalls of child process.`